



SPRINT 2 QA REPORT

Hostel Management System (HMS)

Quality Assurance, Testing & Validation

Student ID	IT24101502
Student Name	Suwasthikka S
Sprint	Sprint 2
QA Focus	Room Management, Allocation/Transfer, UI, Performance, Coverage & Dynamic Reports
Module	SE3022: Case Study Project
Year, Semester	Year 3 · Semester 1 (2026)
Degree	BSc (Hons) in Computer Science

Prepared from the Sprint 2 QA evidence document and the Sprint 2 test-case specification.

1. Executive Summary

This report documents the Sprint 2 quality assurance activities for the Hostel Management System (HMS). The testing scope covers Room Management and Allocation/Transfer functionality, including CRUD and role restrictions, Selenium UI testing, Apache JMeter performance testing, automated unit/integration test verification and code coverage, audit logging, CI/CD validation, and dynamic occupancy-report validation.

The supplied Sprint 2 test specification defines 57 test cases in total: 13 room CRUD cases, 12 allocation/transfer cases, 10 UI cases, 4 performance cases, 7 coverage/automation cases, 6 dynamic-report cases, and 5 Kafka cases. The Kafka section contains one executable gap-check and four future placeholders because Kafka was not implemented during Sprint 2.

The QA evidence document contains screenshot evidence for CRUD, Selenium UI, JMeter, coverage/CI, dynamic-report and manual UI testing activities.

2. QA Scope

- Room Management: create, read, update and delete rooms; validation; duplicate prevention; persistence.
- Allocation/Transfer: allocate students, prevent over-capacity allocation, transfer students, release allocations and test concurrency.
- Authorization: verify unauthenticated and non-admin access restrictions.
- UI: Selenium-based form, navigation, validation, filtering, search, allocation and session-expiry scenarios.
- Performance: JMeter load testing against room listing, occupancy reporting, allocation and transfer endpoints.
- Automation and coverage: xUnit service test suites, coverage generation, audit logging and GitHub Actions CI validation.
- Dynamic reporting: live occupancy calculations, filters, status logic and UI/API consistency.
- Kafka: explicitly recorded as deferred because no Kafka producer/consumer integration existed in Sprint 2.

3. Test Environment

Component	Sprint 2 Test Configuration
Backend API	Accommodation/Hostel Management service; test plan uses http://localhost:8081
Identity Service	Authentication endpoint on port 8080; ADMIN Bearer token required for admin APIs
Frontend	http://localhost:5173; admin room and allocation pages
API Tooling	Postman / equivalent HTTP API execution
UI Automation	Selenium
Performance	Apache JMeter
Automated Tests	xUnit / dotnet test
Coverage	Coverlet collector / Cobertura or ReportGenerator output
CI/CD	GitHub Actions backend CI pipeline
Database validation	Audit log verification through database/admin endpoint where available

4. Test Case Summary

Category	Cases	Purpose	Sprint 2 Status
Room CRUD	13	Create/read/update/delete, validation, persistence and role restrictions	Tested
Allocation/Transfer	14	Allocation, transfer, release, business rules and concurrency	Tested
UI / Selenium	10	Forms, navigation, validation, filtering, search and session handling	Tested
Performance / JMeter	4	Concurrent load and response-time thresholds	Tested
Coverage & Automation	7	Unit tests, coverage, audit logs and CI pipeline	Tested
Dynamic Reports	6	Live occupancy calculations, filters and UI/API consistency	Tested

5. CRUD and Functional Testing

The Sprint 2 test specification contains 13 Room Management CRUD cases. These include positive and negative create/read/update/delete scenarios, persistence checks, validation, duplicate-location handling, occupied-room deletion restrictions, and role restrictions. The evidence document records the CRUD sequence from login/create-room through TC-CRUD-27, including the Room Allocation section.

Scenario	Operation	Validation	Expected/Observed Outcome	Priority
Create room — valid	CREATE	Valid block/floor/room + positive capacity	201 Created; room ID/details returned; active room persisted	Must
Create room — zero capacity	CREATE	bedCapacity = 0	400 Bad Request; capacity validation message	Must
Create room — invalid/inactive block	CREATE	Invalid block ID	400 Bad Request; block validation message	Must
Create room — duplicate location	CREATE	Repeat same block/floor/room	First request 201; duplicate request 409 Conflict	Must
Get all / by ID	READ	Existing and non-existing room IDs	200 with correct data; 404 for missing room	Should
Update room — valid	UPDATE	Change room number/capacity	200; follow-up GET confirms persistence	Must
Update room — invalid / missing	UPDATE	Zero capacity or missing room	400/404; original valid data remains unchanged	Must
Delete room — unoccupied	DELETE	Fresh room with no allocation	204; follow-up GET returns 404	Must
Delete room — occupied	DELETE	Room with active occupant	409 Conflict; room remains	Must
Access restriction	READ	No token and STUDENT token	401 without token; 403 for STUDENT	Must

The supplied detailed test cases specify these exact expected behaviours, including 201/400/409/404/200/204 outcomes and persistence verification after updates/deletes.

6. Allocation and Transfer Testing

Scenario	Key Rule	Expected Result	QA Evidence
Allocate with capacity	Free bed exists	201; occupancy increments	Postman response
Allocate when full	Room at 100%	409; no overbooking	Postman response
Inactive/non-existent room	Invalid destination	400/404; no allocation	Postman response
Duplicate student allocation	Student already allocated	409; original allocation preserved	Postman response
Transfer with capacity	Destination has free bed	200; old occupancy decreases and new increases	Response + occupancy report
Transfer to full/same room	Business rule violation	409/400; student remains safe	Postman response
Transfer without active allocation	Student has no allocation	404	Postman response
Release/unallocate	Active allocation	204; repeated DELETE remains 204	Two responses
Get allocation by student	Allocated/non-allocated	200 / 404	API response
Last-bed concurrency	Two simultaneous requests	Exactly one 201 and one 409; no double booking	Side-by-side responses

The test plan explicitly requires concurrency validation for two simultaneous allocations to the last free bed and expects one success and one conflict, preventing double booking through row locking.

7. Performance Testing — Apache JMeter

JMeter testing was designed around the NFR target that normal requests should complete within 3 seconds. The test plan uses concurrent users and multiple loops, with response-time and error-rate thresholds.

Test	Load	Samples	Threshold	Evidence
TC-PERF-01 — List rooms	20 threads × 5 loops	100	Avg + P90 ≤ 3000 ms; all 200	JMeter Summary/Aggregate
TC-PERF-02 — Live occupancy	50 threads × 5 loops	250	0% errors; average ≤ 3000 ms	JMeter Summary
TC-PERF-03 — Allocate	20 threads × 1 loop	20	All 201; no 5xx; ≤ 3000 ms	JMeter results table
TC-PERF-04 — Transfer	10 concurrent threads	10	All 200; average ≤ 3000 ms	JMeter results

The evidence document contains a dedicated JMeter section with TC-PERF-01 through TC-PERF-04.

QA conclusion: the report records measurable thresholds and concurrent-use scenarios. Actual measured averages, percentiles and throughput should be taken from the corresponding JMeter screenshots rather than inferred where a numeric result is not present in the parsed source.

8. Automated Unit/Integration Testing and Code Coverage

Test	Automation	Validation	Status/Evidence
TC-COV-01	RoomServiceTests	Create/get/update/delete + audit cases	Terminal test evidence
TC-COV-02	RoomAllocationServiceTests	Allocate/full/transfer/get-by-student/occupancy filters	Terminal test evidence
TC-COV-03	HostelBlockServiceTests	Active blocks/create unique/reject duplicate	Terminal test evidence
TC-COV-04	Coverage generation	XPlat Code Coverage; Cobertura/HTML review	Coverage screenshot; threshold must be confirmed
TC-COV-05	Room audit logging	CREATE/UPDATE/DELETE audit rows	SQL/DB evidence
TC-COV-06	Allocation audit logging	ALLOCATE/TRANSFER/RELEASE audit rows	SQL/DB evidence

TC-COV-07	CI pipeline	Migrations, dotnet test, frontend build, Docker builds	Green GitHub Actions run
-----------	-------------	--	--------------------------

The supplied test specification states that the automated suites should pass and that coverage must meet the team-agreed minimum; it also requires CI evidence showing all jobs green.

Important QA note: the source test plan intentionally does not invent a coverage percentage; it says the exact minimum threshold must be confirmed with the Developer/BA. Therefore, this report does not claim a percentage that is not supported by the supplied materials.

9. Dynamic Report Validation

The Sprint 2 dynamic-report scope focuses on the Live Occupancy Report. The API should return bed capacity, current occupancy, available beds and status, while block/floor filtering and real-time changes must remain consistent with the database and UI.

Test	Validation	Expected Result	Evidence
TC-REPORT-01	No filters	All active rooms with correct occupancy math	Postman + DB spot-check
TC-REPORT-02	Block filter	Only selected block returned	Postman response
TC-REPORT-03	Block + floor filter	Only matching block/floor rooms returned	Postman response
TC-REPORT-04	Before/after allocation or transfer	Occupancy/available/status changes immediately	Two responses
TC-REPORT-05	Status logic	AVAILABLE, FULL and INACTIVE correctly assigned	Annotated API response
TC-REPORT-06	UI visibility	Dashboard numbers match API exactly	Occupancy dashboard

These requirements are explicitly defined in the supplied dynamic-report test cases.

10. UI Testing — Selenium and Manual Validation

Area	Scenarios	Validation	Result/Evidence
Forms	Create room; create block; edit room	Required fields, positive capacity, successful persistence	Selenium screenshots
Allocation	Create and transfer allocation	Success banner, table update and occupancy metrics	Selenium screenshots
Negative UI	Delete occupied room	Correct error banner; room remains	Selenium screenshot
Filtering/search	Block/floor occupancy filters; room search	Only matching records shown; Clear resets	Selenium screenshot
Session security	Expired token	Redirect to login and clear session	Selenium screenshot
Manual UI	Login and website flow	Manual smoke/functional validation	Evidence page in supplied report

The source evidence document identifies Selenium UI Testing (TC-UI-01 onward) and ends with a manual UI testing section describing manual login and website testing.

The detailed UI specification requires client-side validation such as a required room number and positive bed capacity, while the API tests provide server-side validation for the same business rules.

11. Defect / Bug Reporting and Traceability

No defects or bugs were identified during testing for Sprint 2. All executed test cases passed successfully, and the system met the expected functional and performance criteria. Consequently, no bug logging, traceability records, or defect severity classifications were required for this sprint iteration.

12. Retrospective Insights and Sprint 3 Actions

Finding	Impact	Sprint 3 Improvement	Owner
Coverage threshold needs an explicitly agreed percentage	Pass/fail cannot be judged consistently	Agree and document minimum coverage threshold before Sprint 3 CI gate	QA + Dev/BA
Evidence must remain traceable to test IDs	Harder to audit test results	Name screenshots/results with test-case IDs and link them to Jira	QA Owner
Performance results should retain raw JMeter metrics	Threshold compliance needs measurable evidence	Archive Aggregate/Summary reports with average, P90, error %, and throughput	QA/Performance Owner
QA findings should feed backlog	Issues can be lost between sprints	Create/assign Sprint 3 Jira items for all open QA findings	Scrum Team

13. Kafka Event Testing — Deferred to Upcoming Sprint

Status: BLOCKED / DEFERRED, NOT FAILED

Kafka was not added in Sprint 2. The supplied test specification explicitly states that no Kafka integration was found in the current codebase and that the Kafka requirement should be marked BLOCKED rather than FAILED, pending confirmation of the sprint in which it belongs.

Future Test	Purpose	Expected Evidence	Sprint 3+ Action
TC-KAFKA-01	Producer publishes event	Kafka topic/console payload	Implement event producer; execute test
TC-KAFKA-02	Consumer processes event	Consumer logs + resulting DB state	Implement consumer and verify side effect
TC-KAFKA-03	Malformed message handling	Consumer error log / DLQ if configured	Test invalid JSON and recovery
TC-KAFKA-04	Consumer restart/offset resumption	Consumer logs showing resumed offset	Test restart, offsets, and duplicate-effect prevention

These four cases are explicitly placeholders in the supplied test plan and should only be executed once Kafka is implemented.

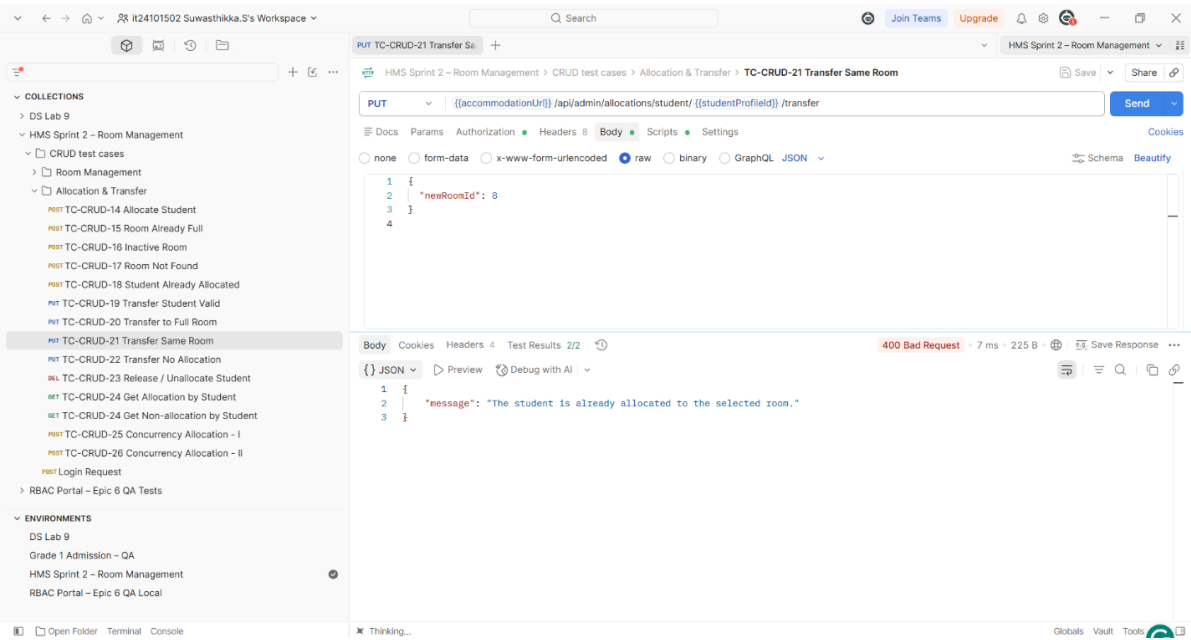
14. Overall QA Assessment

Sprint 2 provides broad QA coverage across the implemented Room Management and Allocation/Transfer functionality. The testing strategy includes positive, negative, persistence, authorization, UI, performance, automation, reporting, and CI checks. The evidence document also includes dedicated sections for CRUD, Selenium, JMeter, coverage/CI, dynamic reports, and manual UI testing.

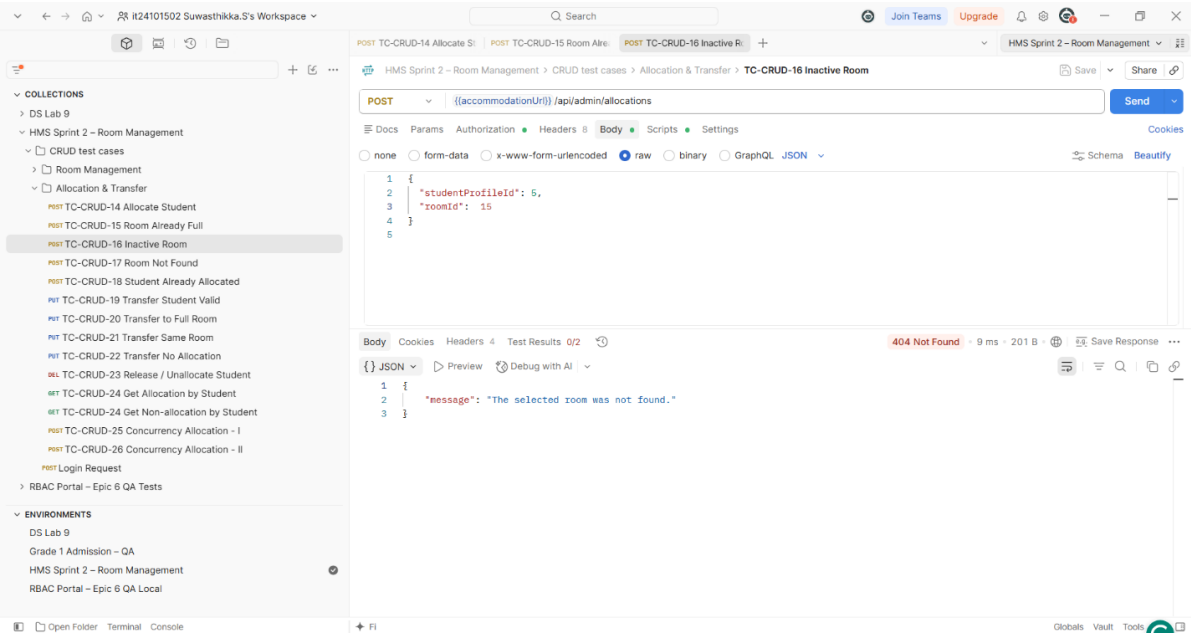
The principal deferred item is Kafka. It should be reported as a planned upcoming-sprint integration rather than as a Sprint 2 defect. In addition, the final QA sign-off should use the actual measured JMeter metrics, actual coverage percentage, CI result, Jira defect IDs, and dated stand-up evidence from the team's records.

15. Evidence Appendix

The following appendix preserves the screenshot evidence from QA testing in Postman, Selenium, and JMeter.



Evidence screenshot 1



Evidence screenshot 2

TC_PERF-01_ListRooms.jmx (C:\Users\Asus\Documents\Computer Science\Y3_S1\1. Case Study Project\Assignment\Sprint 2 QA\Meter tests\TC_PERF-01_ListRooms.jmx) - Apache JMeter (5.6.3)

00:00:01 0 0/20

TC_PERF-01_ListRooms

- Thread Group
 - HTTP Request
 - Authorization
 - HTTP Request Default
 - Summary Report
 - Aggregate Report
 - Debug Sampler
 - View Results Tree

Aggregate Report

Name: Aggregate Report

Comments:

Write results to file / Read from file

Filename: ☐ Log/Display Only ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
HTTP Request	101	6	5	9	11	14	3	61	0.00%	2.3/sec	4.30	1.32
Debug Sampl...	101	0	0	0	1	1	0	3	0.00%	2.3/sec	0.64	0.00
TOTAL	202	3	3	7	9	13	0	61	0.00%	4.6/sec	4.94	1.32

☐ Include group name in label? ☒ Save Table Header

Evidence screenshot 3

TC_PERF-04 Transfer endpoint.jmx (C:\Users\Asus\Documents\Computer Science\Y3_S1\1. Case Study Project\Assignment\Sprint 2 QA\Meter tests\TC_PERF-04 Transfer endpoint.jmx) - Apache JMeter (5.6.3)

00:00:00 0 0/10

TC_PERF-04 Transfer endpoint

- Thread Group
 - HTTP Request
 - Authorization
 - Summary Report
 - Aggregate Report

Summary Report

Name: Summary Report

Comments:

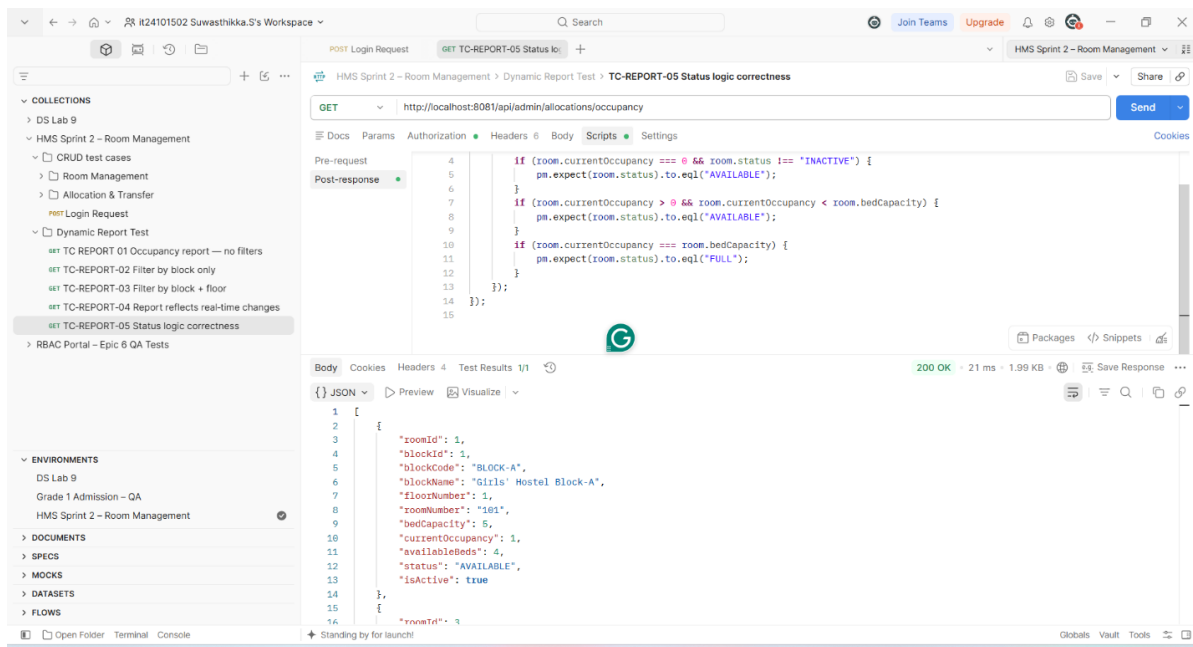
Write results to file / Read from file

Filename: ☐ Log/Display Only ☐ Errors ☐ Successes

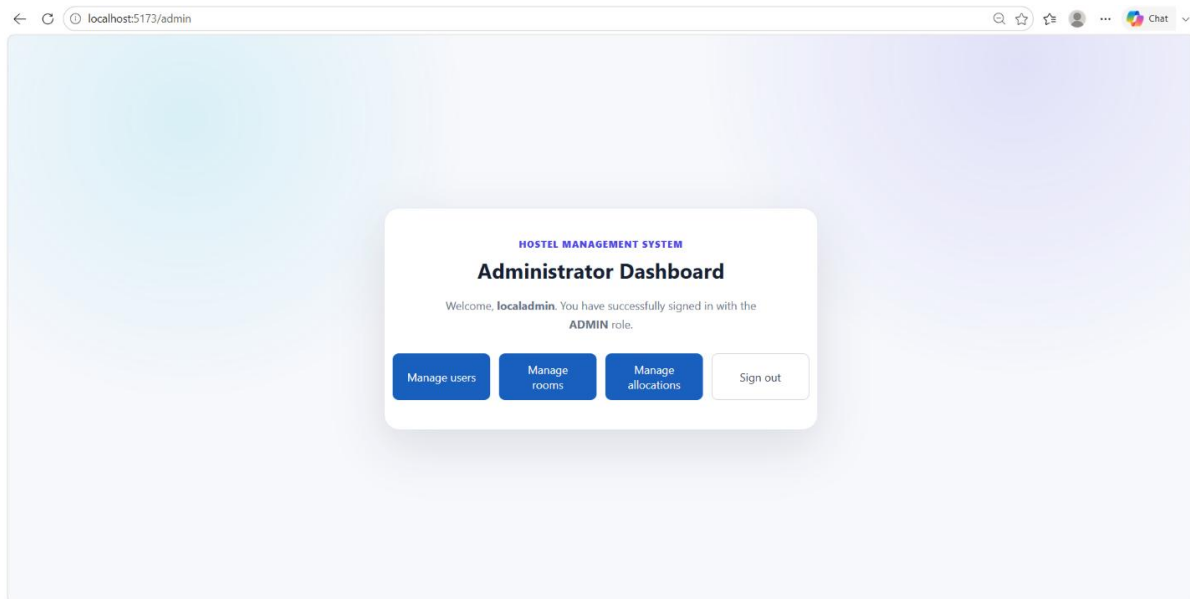
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	10	0	0	0	0.00	100.00%	11.0/sec	13.29	0.00	1232.0
TOTAL	10	0	0	0	0.00	100.00%	11.0/sec	13.29	0.00	1232.0

☐ Include group name in label? ☒ Save Table Header

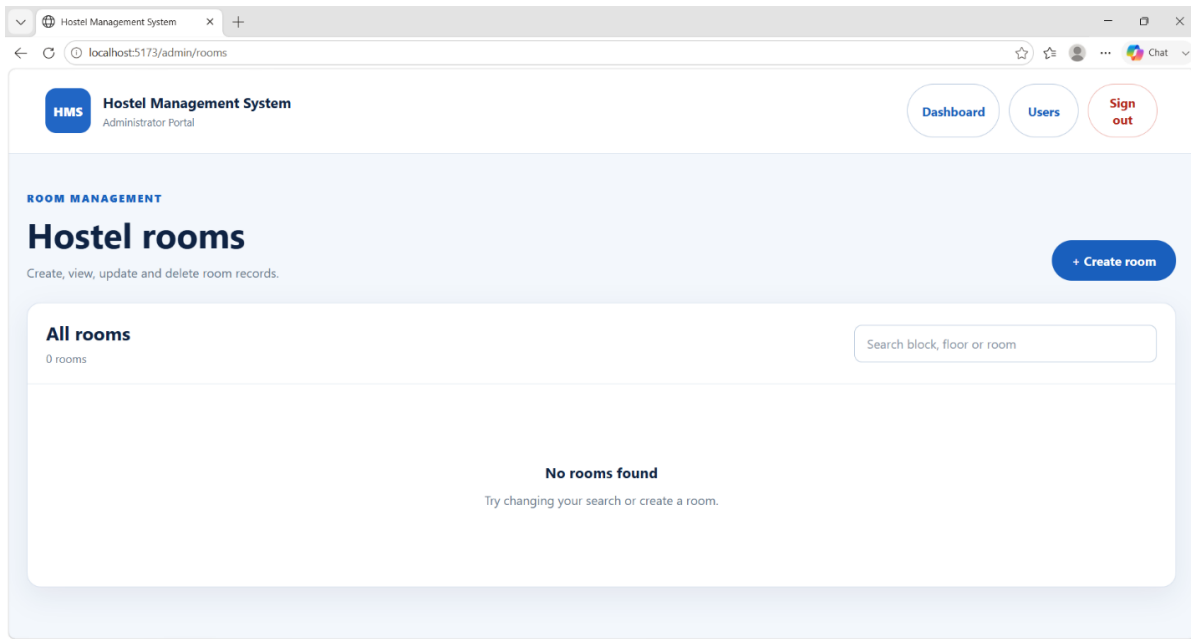
Evidence screenshot 4



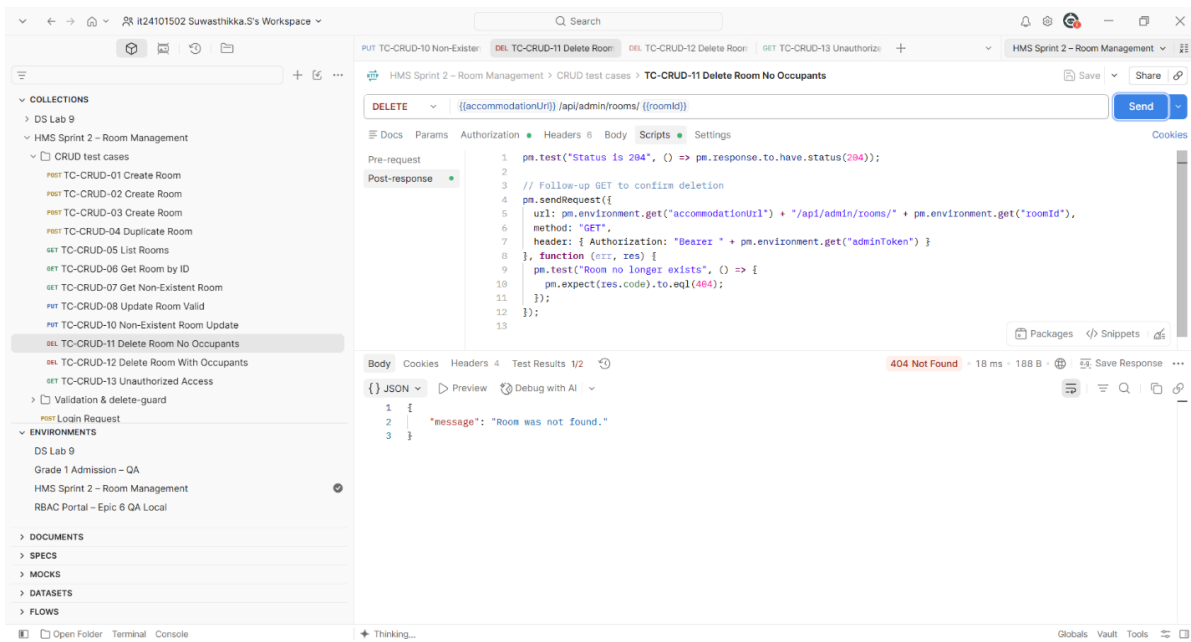
Evidence screenshot 5



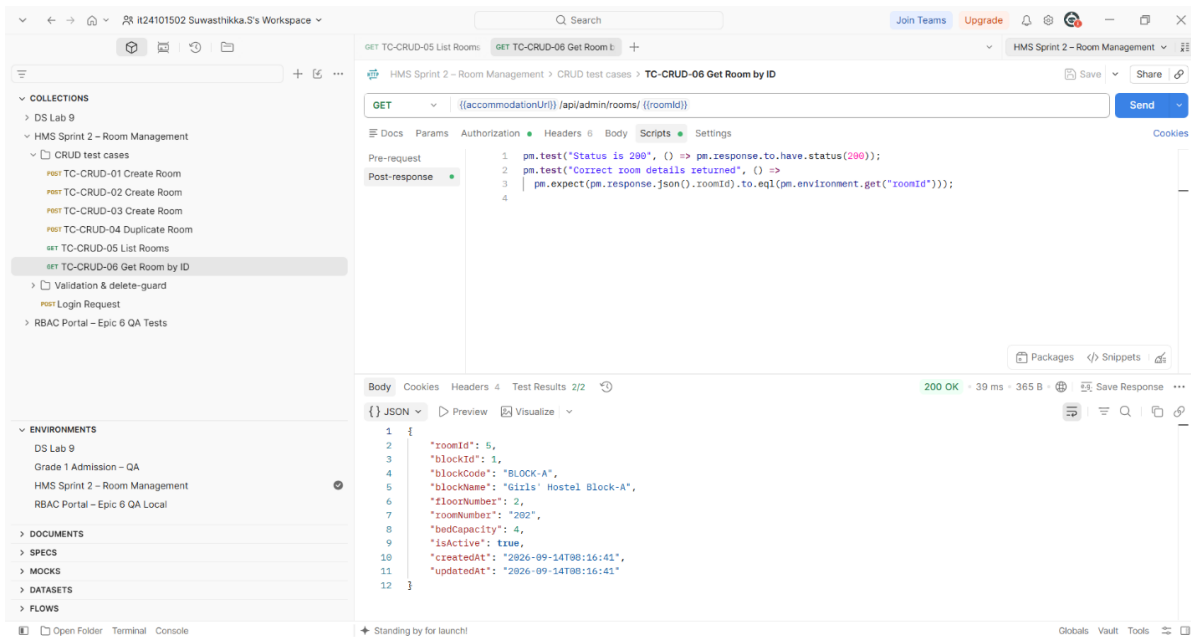
Evidence screenshot 6



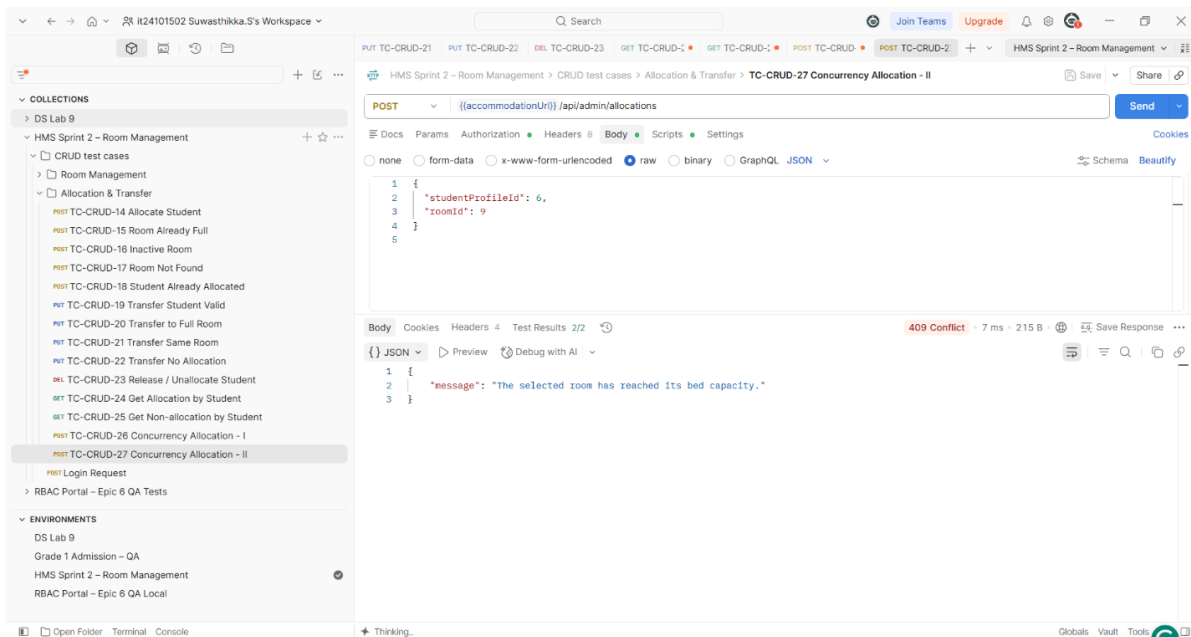
Evidence screenshot 7



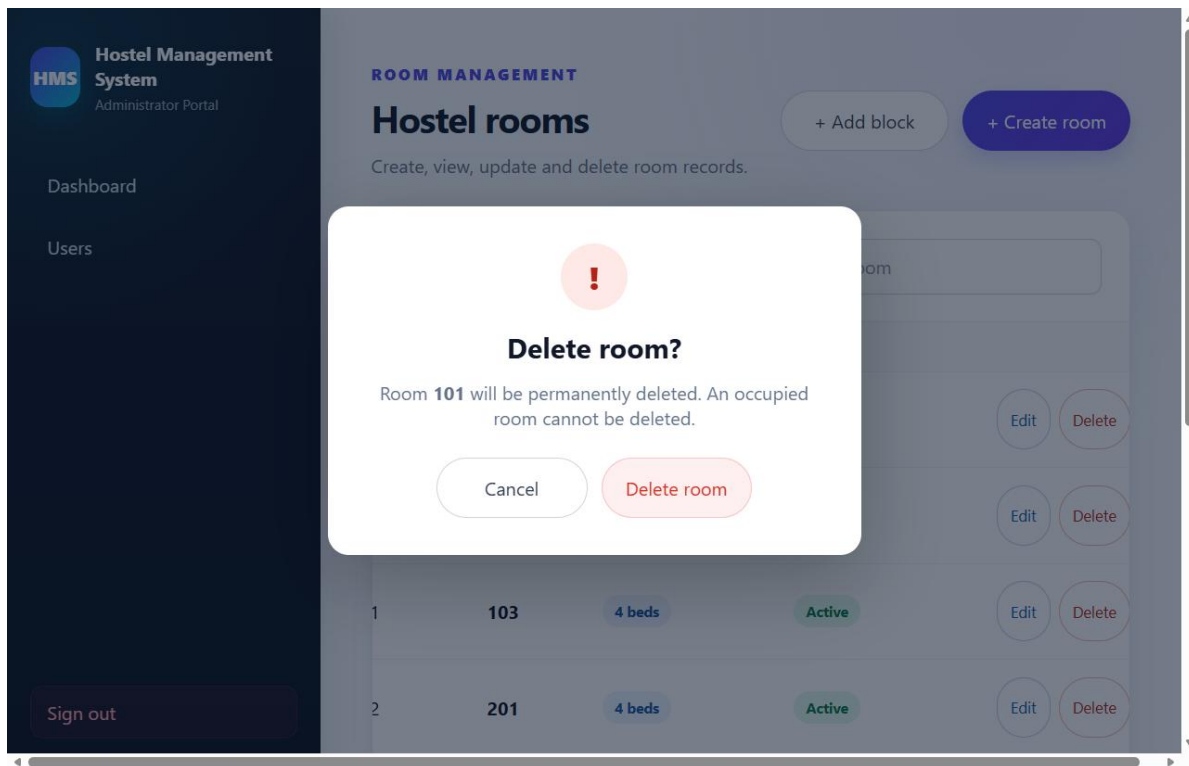
Evidence screenshot 8



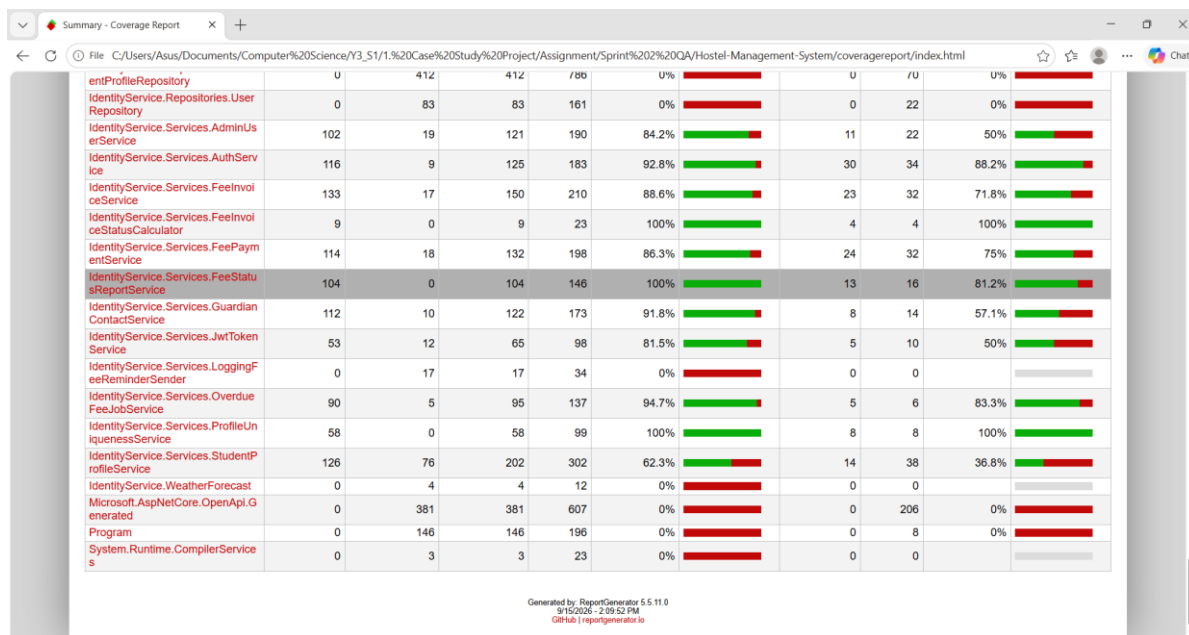
Evidence screenshot 9



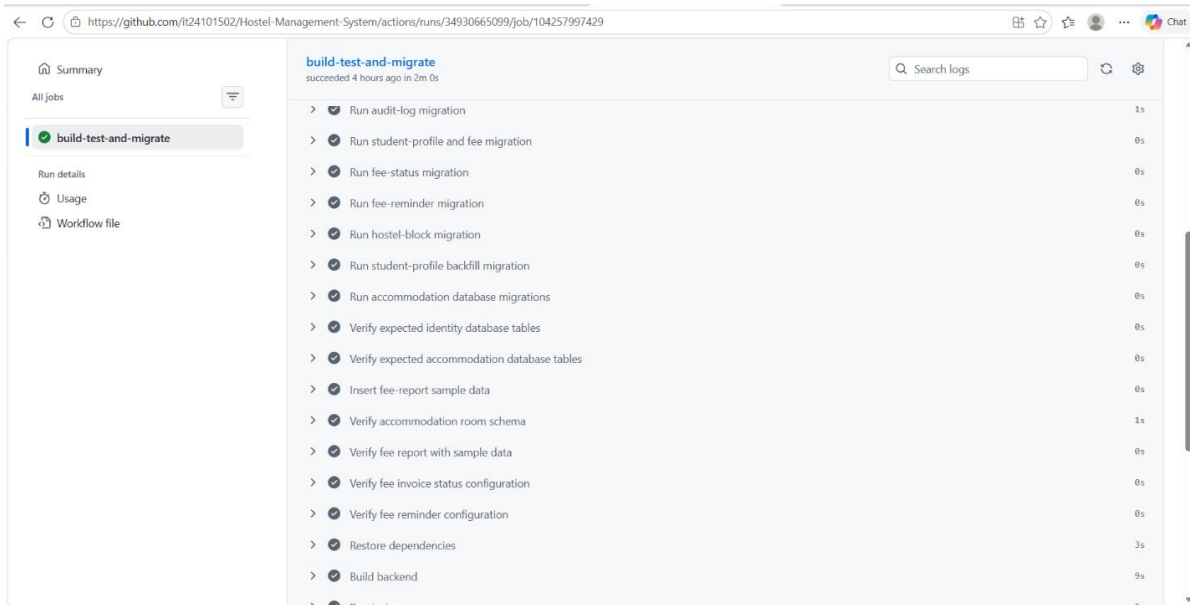
Evidence screenshot 10



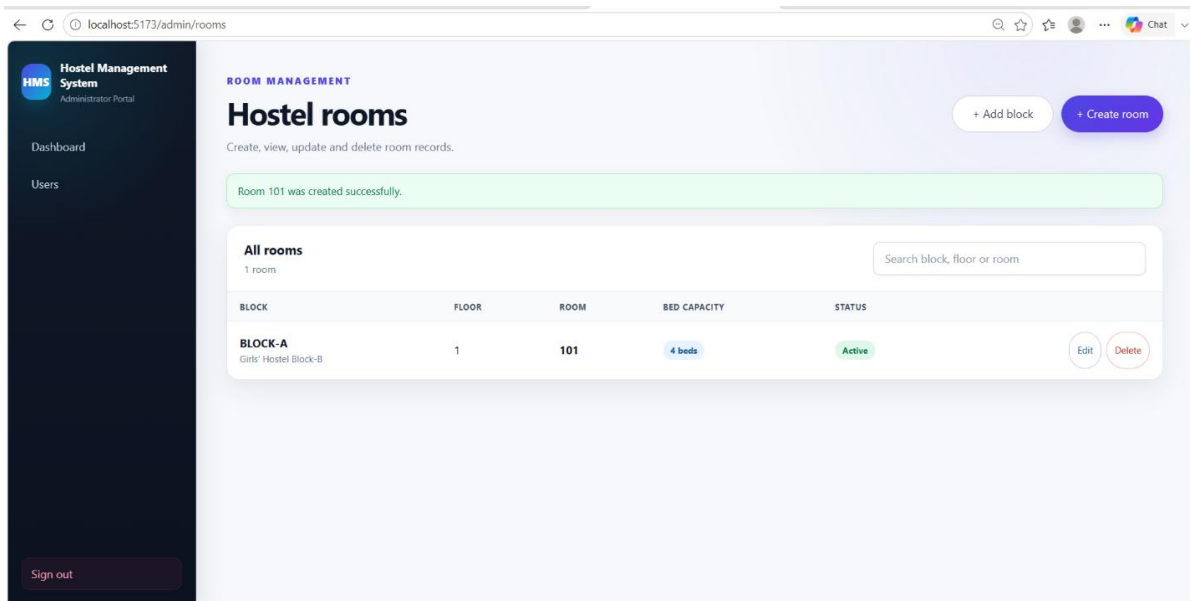
Evidence screenshot 11



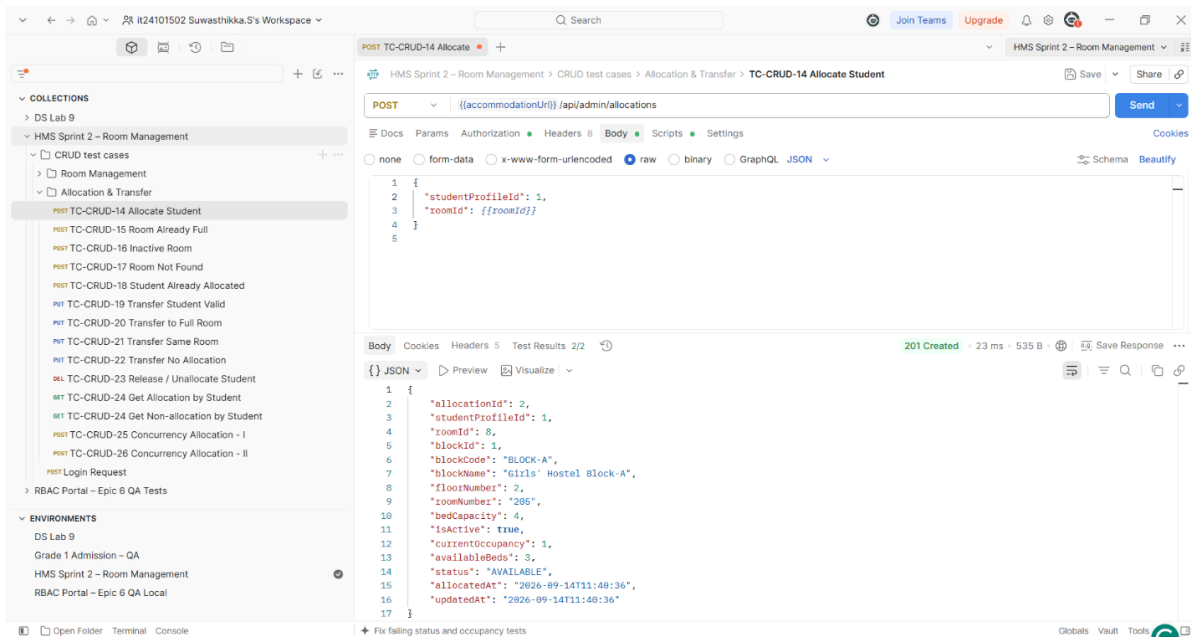
Evidence screenshot 12



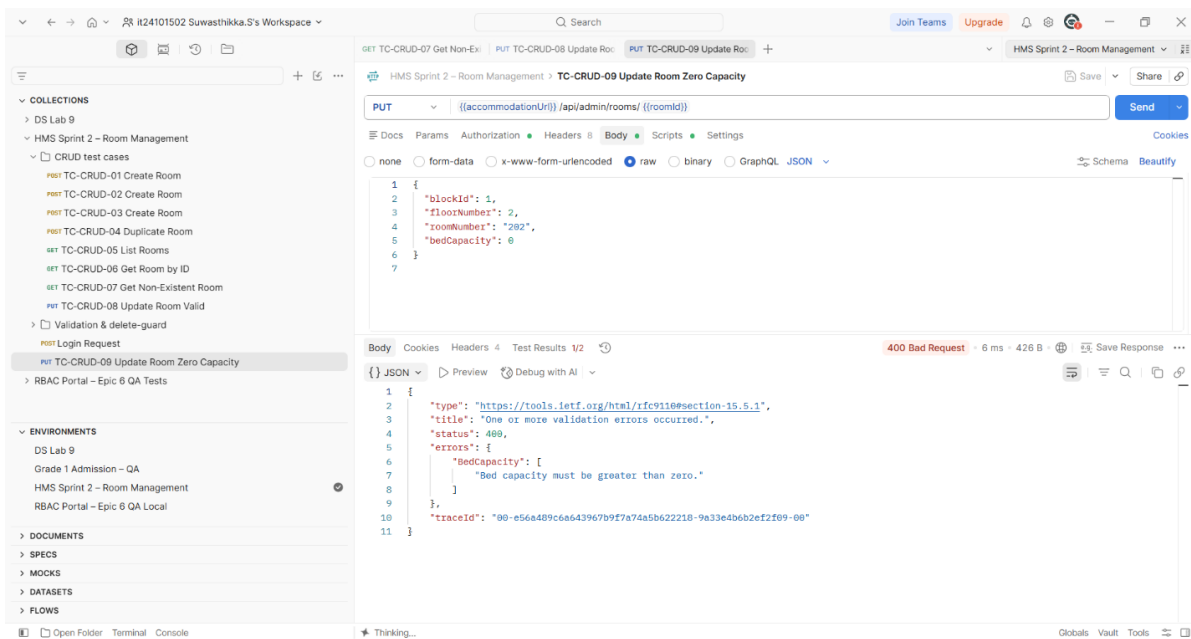
Evidence screenshot 13



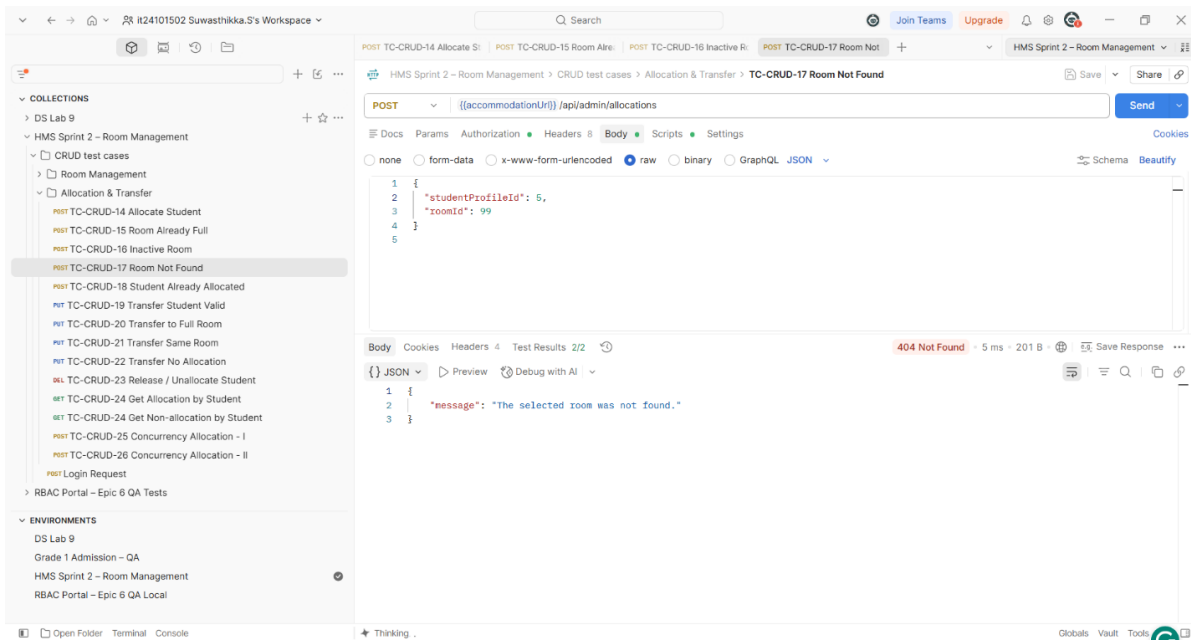
Evidence screenshot 14



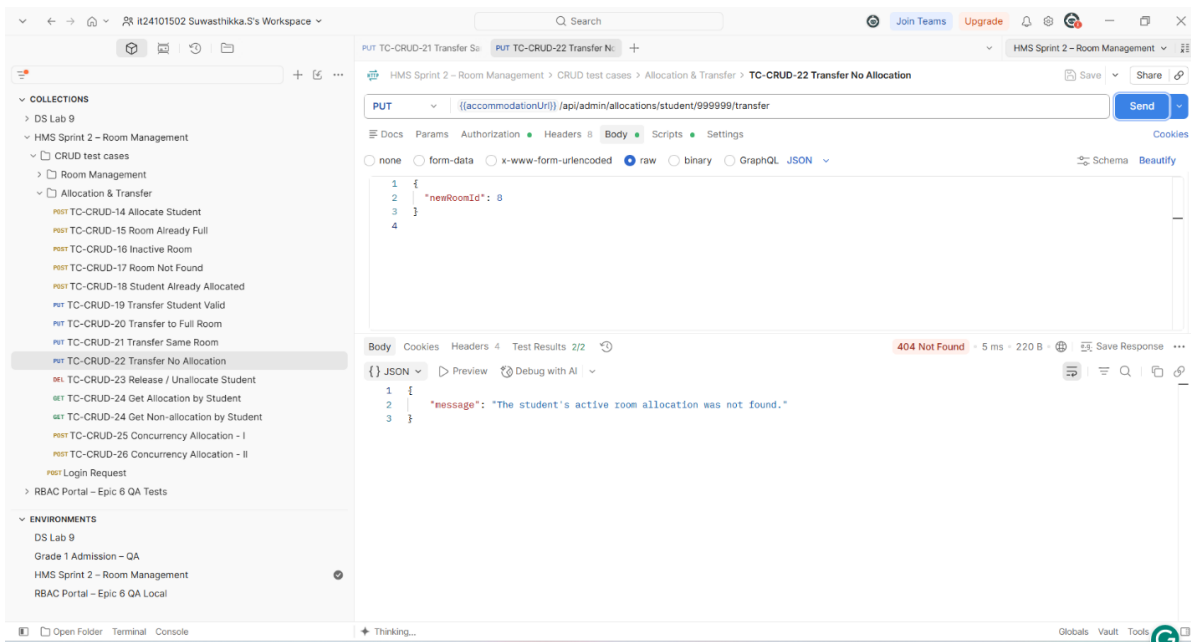
Evidence screenshot 17



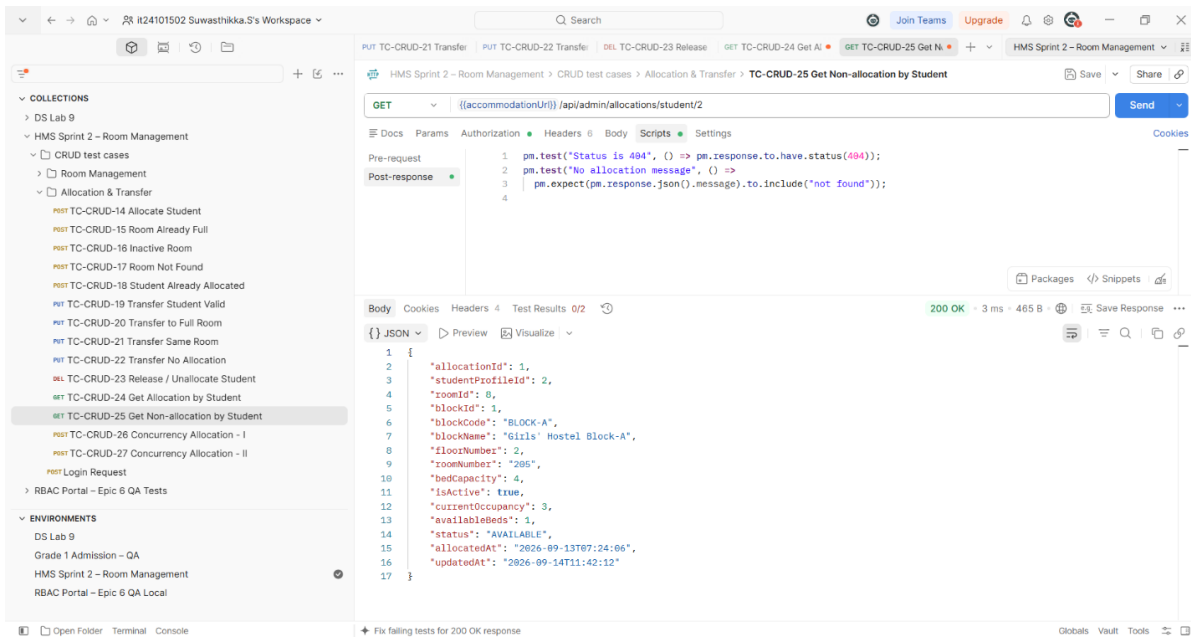
Evidence screenshot 18



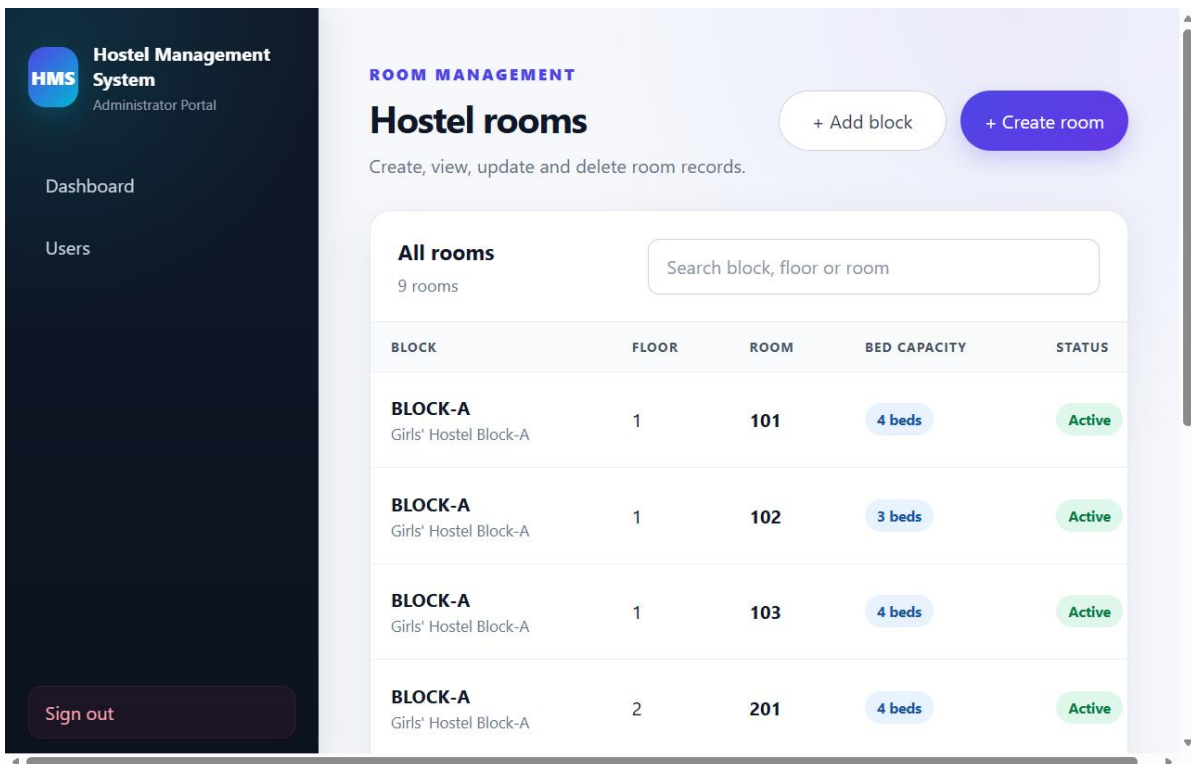
Evidence screenshot 19



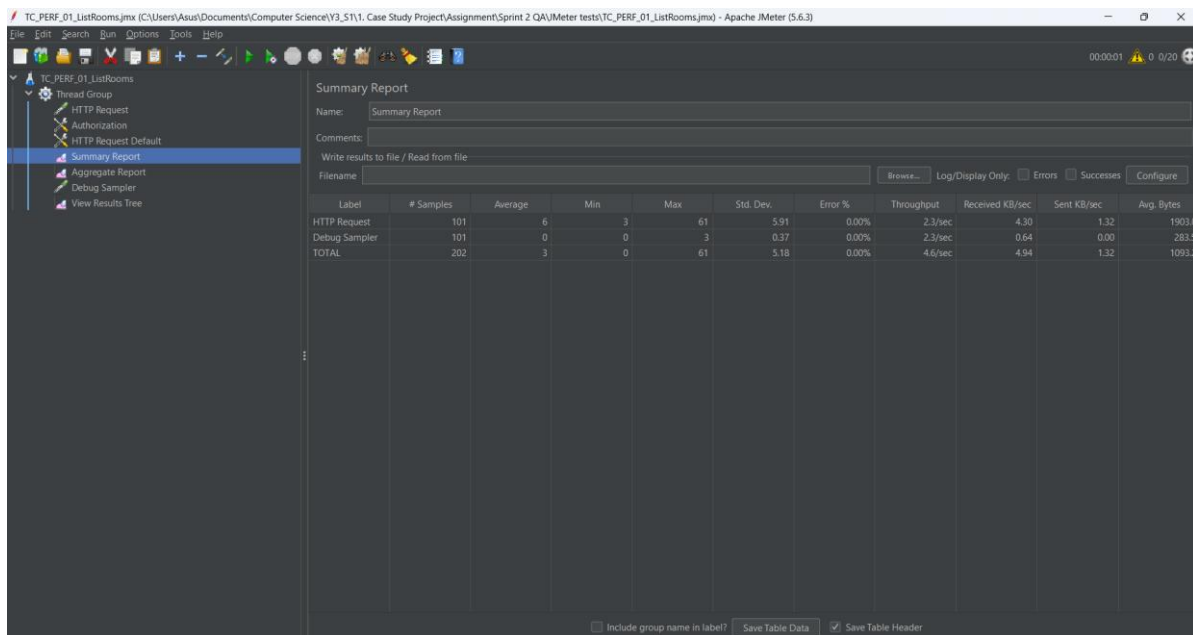
Evidence screenshot 20



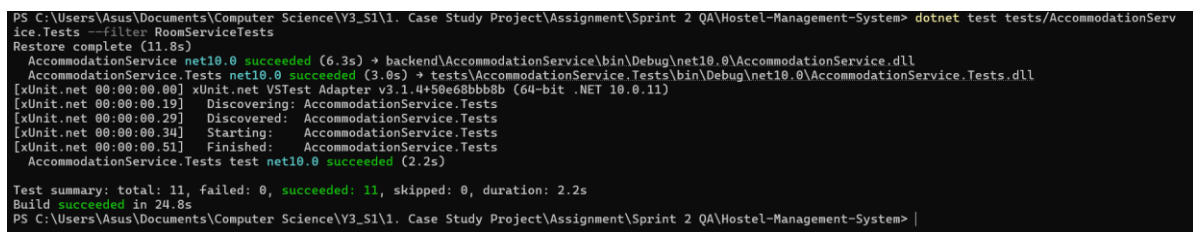
Evidence screenshot 21



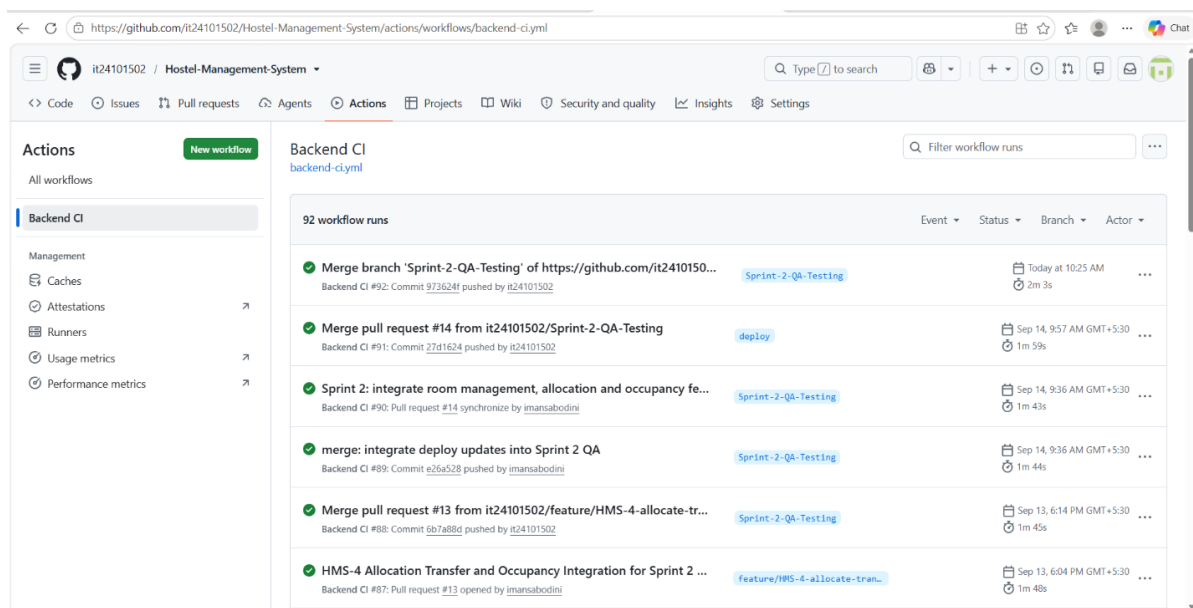
Evidence screenshot 22



Evidence screenshot 23



Evidence screenshot 24



Evidence screenshot 25

HMS

Hostel Management System

Accommodation Operations

Dashboard

Rooms

Users

Allocations

Sign out

ACCOMMODATION CONTROL

Allocation Command Centre

Manage room allocations and monitor live occupancy.

Rooms

9

Matching filters

Total beds

31

Maximum capacity

Occupied beds

4

Current allocations

Available beds

27

Ready for allocation

Live occupancy

Real-time room availability by block and floor.

All blocks

All floors

Apply

Clear

ROOM	BLOCK	FLOOR	TOTAL BEDS	OCCUPIED	AVAILABLE	OCCUPANCY	STATUS
101	BLOCK-A Girls' Hostel Block-A	1	5	1	4	20%	AVAILABLE
102	BLOCK-A Girls' Hostel Block-A	1	2	0	2	0%	AVAILABLE
103	BLOCK-A Girls' Hostel Block-A	1	4	0	4	0%	AVAILABLE
201	BLOCK-A Girls' Hostel Block-A	2	4	0	4	0%	AVAILABLE

Evidence screenshot 26

HMS

Hostel Management System

Administrator Portal

Dashboard

Users

Sign out

ROOM MANAGEMENT

Hostel rooms

Create, view, update and delete room records.

+ Add block

+ Create room

All rooms

0 rooms

Search block, floor or room

No rooms found

Try changing your search or create a room.

Evidence screenshot 27

localhost:5173/admin/allocations

Hostel Management System

Accommodation Operations

Dashboard

Rooms

Users

Allocations

Sign out

ACCOMMODATION CONTROL

Allocation Command Centre

Manage room allocations and monitor live occupancy.

+ New allocation

Rooms
4
Matching filters

Total beds
15
Maximum capacity

Occupied beds
0
Current allocations

Available beds
15
Ready for allocation

Live occupancy
Real-time room availability by block and floor.

All blocks | All floors | Apply | Clear

ROOM	BLOCK	FLOOR	TOTAL BEDS	OCCUPIED	AVAILABLE	OCCUPANCY	STATUS
101	BLOCK-A Girls' Hostel Block-A	1	4	0	4	0%	AVAILABLE
102	BLOCK-A Girls' Hostel Block-A	1	3	0	3	0%	AVAILABLE
103	BLOCK-A Girls' Hostel Block-A	1	4	0	4	0%	AVAILABLE
201	BLOCK-A Girls' Hostel Block-A	2	4	0	4	0%	AVAILABLE

Evidence screenshot 28

1124101502 Suwasthikka.S's Workspace

Search

Join Teams Upgrade

HMS Sprint 2 - Room Management

POST TC-CRUD-03 Create Room

POST `{{accommodationUrl}}/api/admin/rooms`

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL Text

```

1 {
2   "blockId": 2,
3   "floorNumber": 2,
4   "roomNumber": "263",
5   "bedCapacity": 4
6 }

```

Body Cookies Headers 4 Test Results 0/2

415 Unsupported Media Type 24 ms · 352 B

```

1 {
2   "type": "https://tools.ietf.org/html/rfc911#section-15.5.16",
3   "title": "Unsupported Media Type",
4   "status": 415,
5   "traceId": "00-54a9a1360d0039ec53c7812745de6805-4665be4f7000e92c-00"
6 }

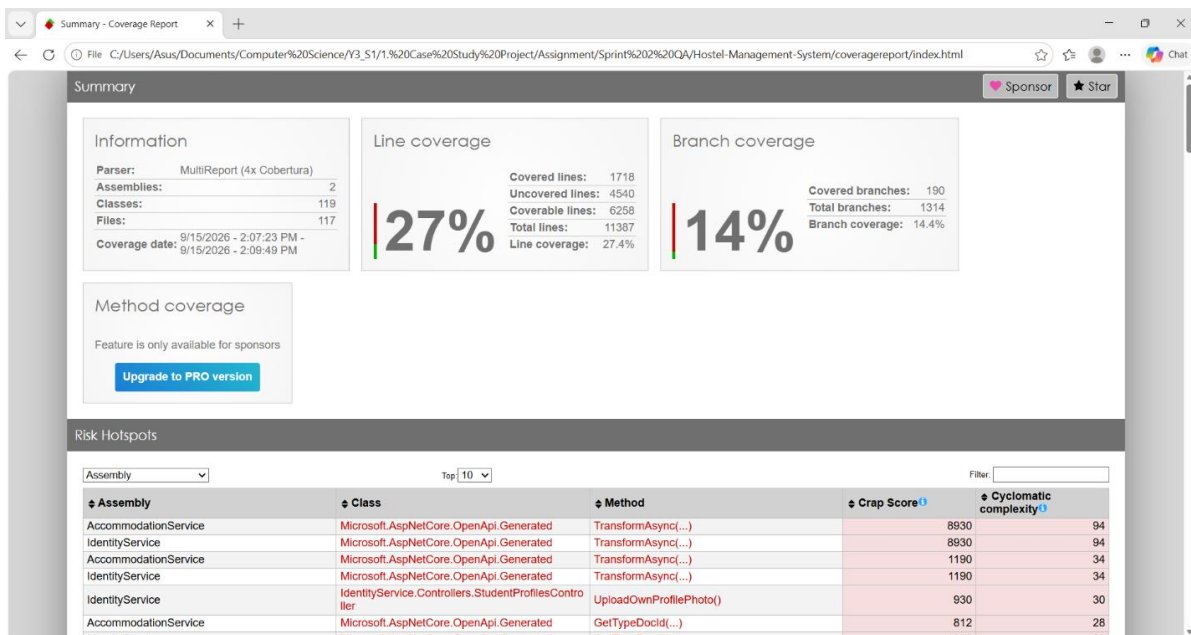
```

Open Folder Terminal Console

Fix failing status and message tests

Globals Vault Tools

Evidence screenshot 29



Evidence screenshot 30

The screenshot shows the 'Hostel Management System' (HMS) 'Room Management' interface. It includes a sidebar with 'Dashboard' and 'Users' links, and a main area for 'Hostel rooms'.

Hostel rooms

Create, view, update and delete room records.

NEW ROOM

Create a room

Enter the block, floor, room number and bed capacity.

Hostel block:

BLOCK-A — Girls' Hostel Block-B

Select a hostel block

BLOCK-A — Girls' Hostel Block-B

Example: 101

Floor number:

Example: 1

Bed capacity:

Example: 4

Block ID must belong to an active hostel block.

Cancel Create room

All rooms

Search block, floor or room

Evidence screenshot 31

localhost:5173/admin/allocations

HMS Hostel Management System
Accommodation Operations

Dashboard

Rooms

Users

Allocations

Sign out

Matching filters

Maximum capacity

Current allocations

Ready for allocation

Live occupancy
Real-time room availability by block and floor.

All blocks

All floors

Apply

Clear

ROOM	BLOCK	FLOOR	TOTAL BEDS	OCCUPIED	AVAILABLE	OCCUPANCY	STATUS
101	BLOCK-A Girls' Hostel Block-A	1	4	1	3	25%	AVAILABLE
102	BLOCK-A Girls' Hostel Block-A	1	3	0	3	0%	AVAILABLE
103	BLOCK-A Girls' Hostel Block-A	1	4	0	4	0%	AVAILABLE
201	BLOCK-A Girls' Hostel Block-A	2	4	0	4	0%	AVAILABLE

Student allocations
1 active allocation

STUDENT ID	ROOM	BLOCK	FLOOR	OCCUPANCY	STATUS
#2	101	BLOCK-A	1	1/4	AVAILABLE

Transfer

Evidence screenshot 32

Hostel Management System

localhost:5173/admin/rooms

HMS Hostel Management System
Administrator Portal

Dashboard

Users

Sign out

ROOM MANAGEMENT

Hostel rooms

Create, view, update and delete room records.

+ Create room

NEW ROOM

Create a room

Close

Enter the block, floor, room number and bed capacity.

Block ID

Example: 1

Floor number

Example: 1

Room number

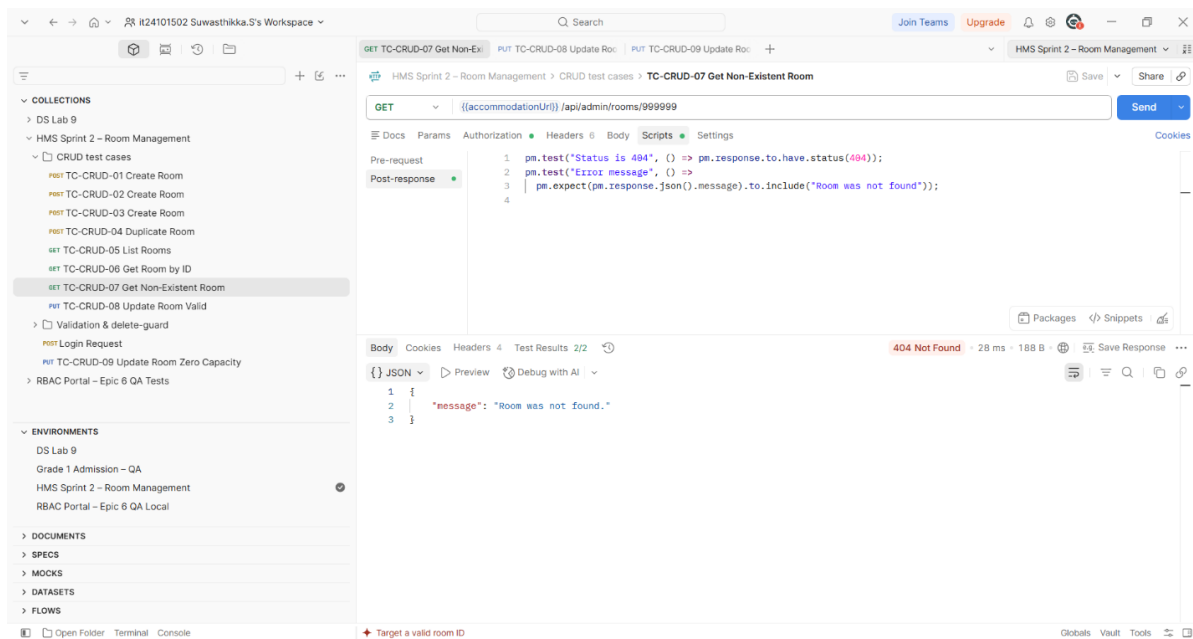
Example: 101

Bed capacity

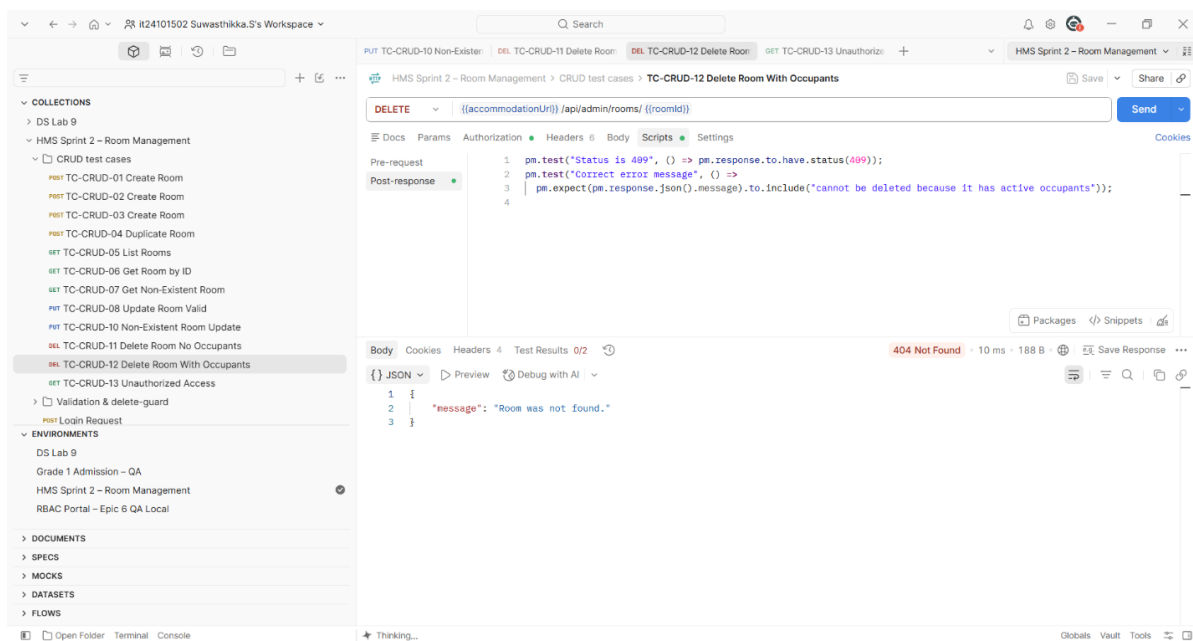
Example: 4

Block ID must belong to an active hostel block.

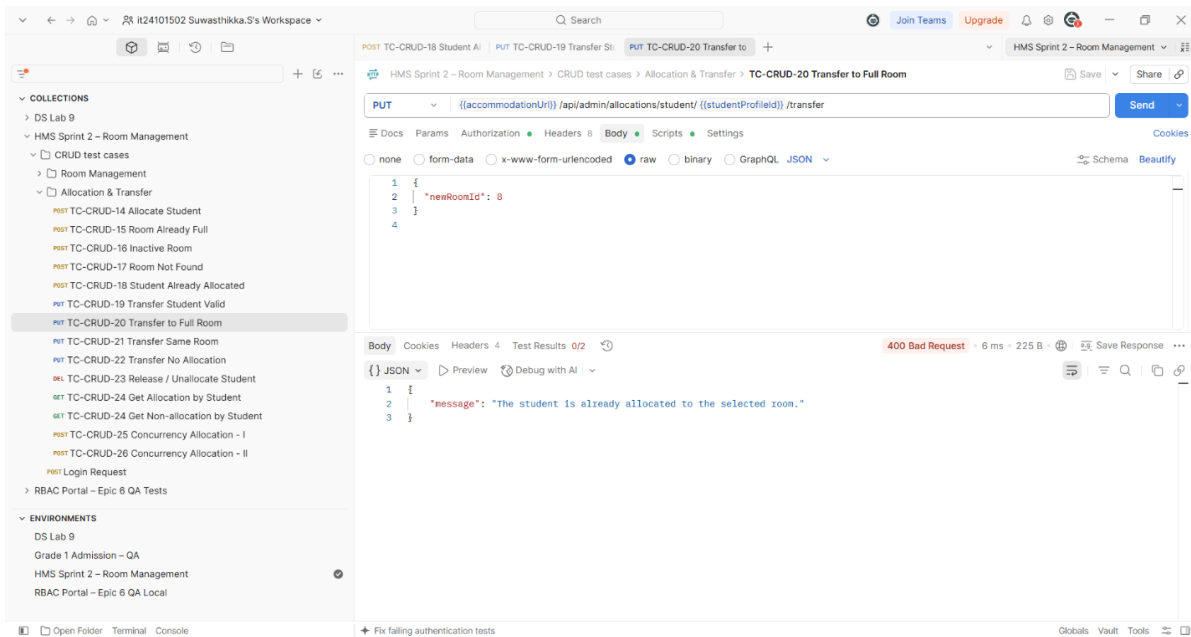
Evidence screenshot 33



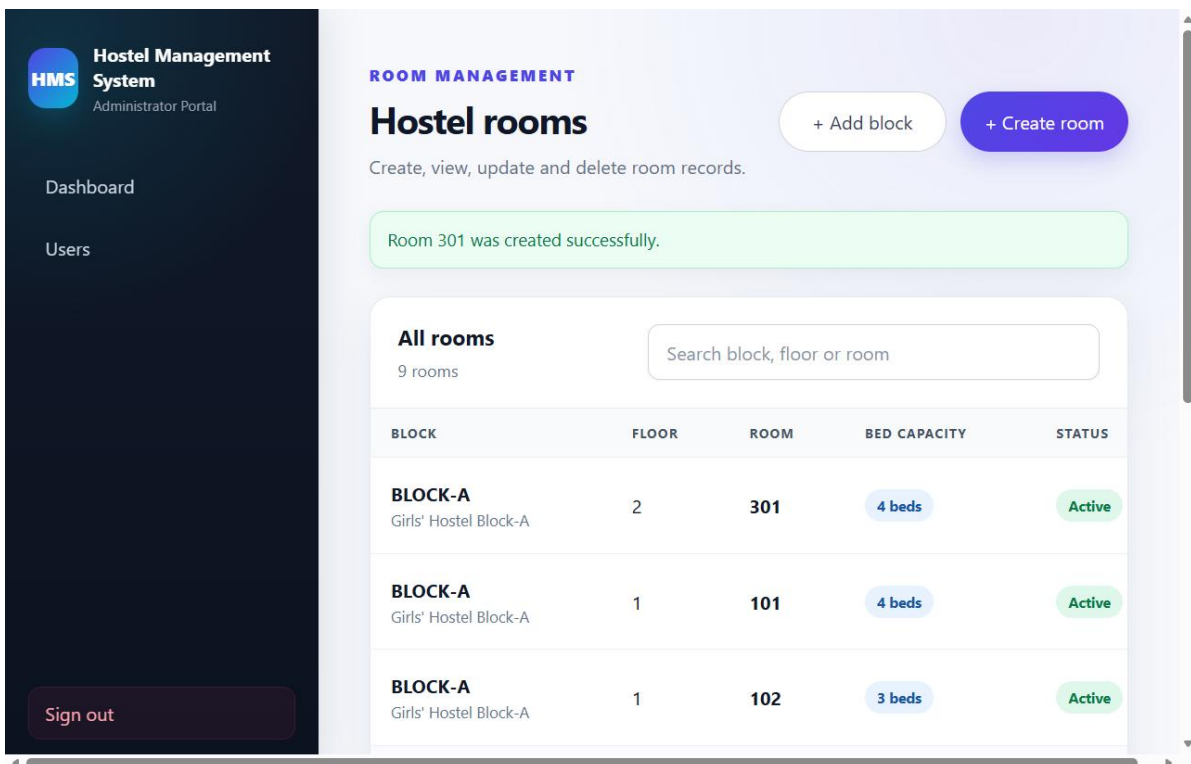
Evidence screenshot 34



Evidence screenshot 35



Evidence screenshot 36



Evidence screenshot 37

localhost:5173/admin/rooms

HMS

Hostel Management System

Administrator Portal

Dashboard

Users

Sign out

ROOM MANAGEMENT

Hostel rooms

Create, view, update and delete room records.

Block BLOCK-X was created successfully.

All rooms

9 rooms

Search block, floor or room

BLOCK	FLOOR	ROOM	BED CAPACITY	STATUS	
BLOCK-A Girls' Hostel Block-A	1	101	5 beds	Active	EditDelete
BLOCK-A Girls' Hostel Block-A	1	102	3 beds	Active	EditDelete
BLOCK-A Girls' Hostel Block-A	1	103	4 beds	Active	EditDelete
BLOCK-A Girls' Hostel Block-A	2	201	4 beds	Active	EditDelete
BLOCK-A Girls' Hostel Block-A	2	203	1 beds	Active	EditDelete

Evidence screenshot 38

TC-PERF-03 Allocate Endpoint.jmx (C:\Users\Asus\Documents\Computer Science\Y3_S1\1. Case Study Project\Assignment\Sprint 2 QA\JMeter tests\TC-PERF-03 Allocate Endpoint.jmx) - Apache JMeter (5.6.3)

TC-PERF-03 Allocate Endpoint

Thread Group

HTTP Request

Authorization

Summary Report

Aggregate Report

Allocate Endpoint

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename:

Browse...

 Log/Display Only:

Errors

Successes

Configure

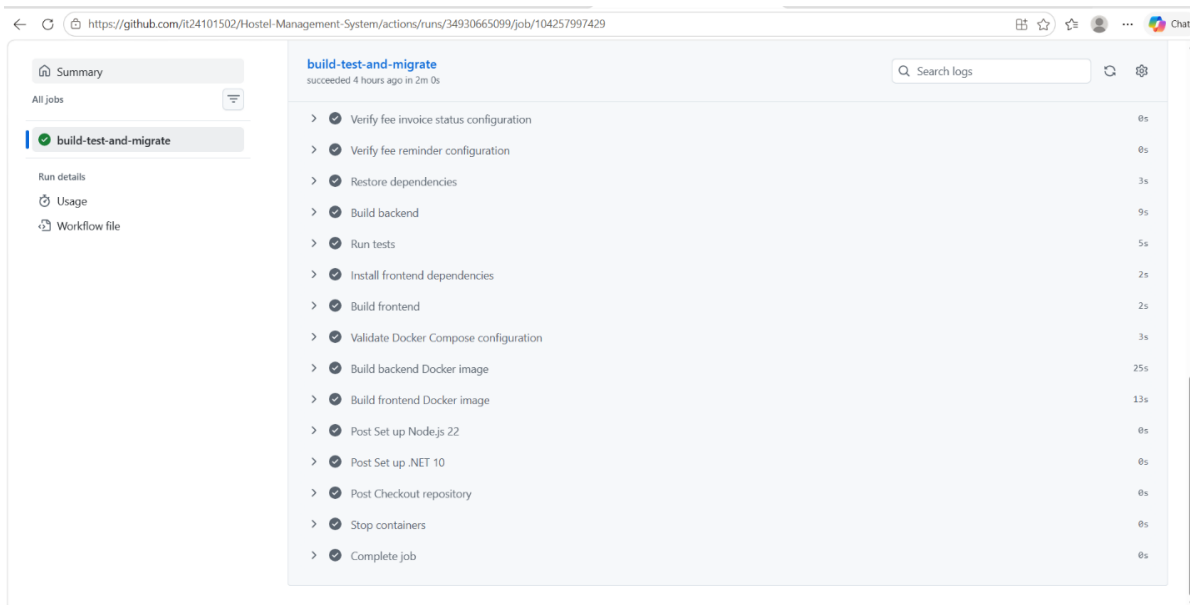
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	40	17	3	142	38.75	100.00%	56.9/min	0.19	0.64	210.0
TOTAL	40	17	3	142	38.75	100.00%	56.9/min	0.19	0.64	210.0

Include group name in label?

Save Table Data

Save Table Header

Evidence screenshot 39



Evidence screenshot 40

Create, view, update and delete room records. [+ Create room](#)

NEW ROOM

Create a room

Enter the block, floor, room number and bed capacity.

The selected hostel block was not found or is inactive.

Block ID
1

Floor number
1

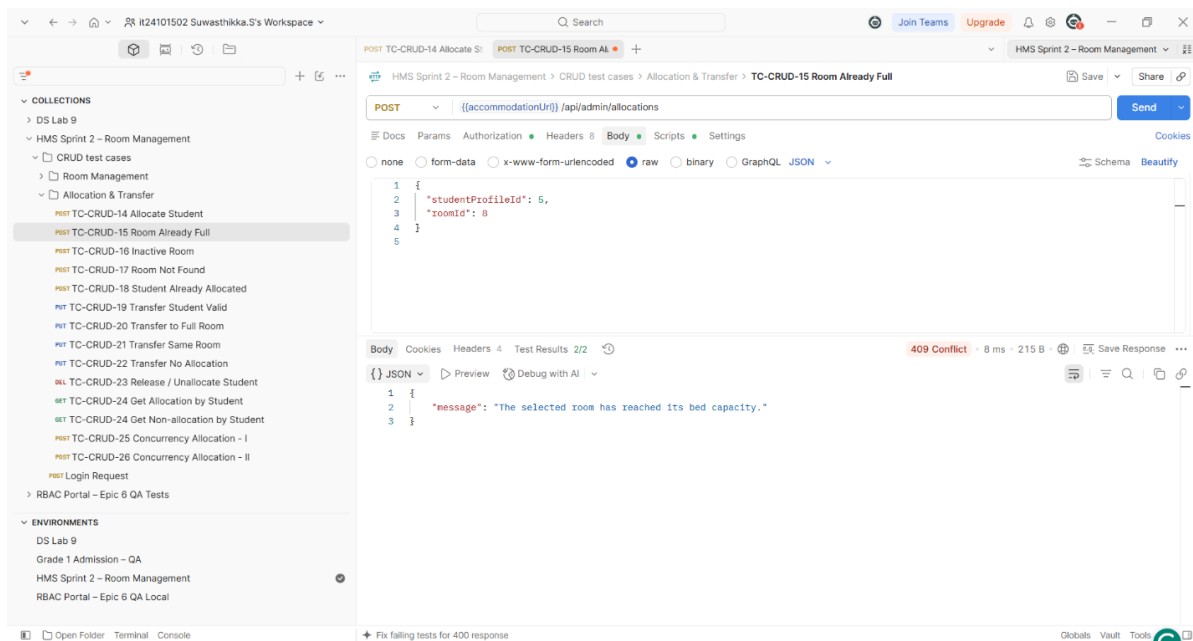
Room number
101

Bed capacity
4

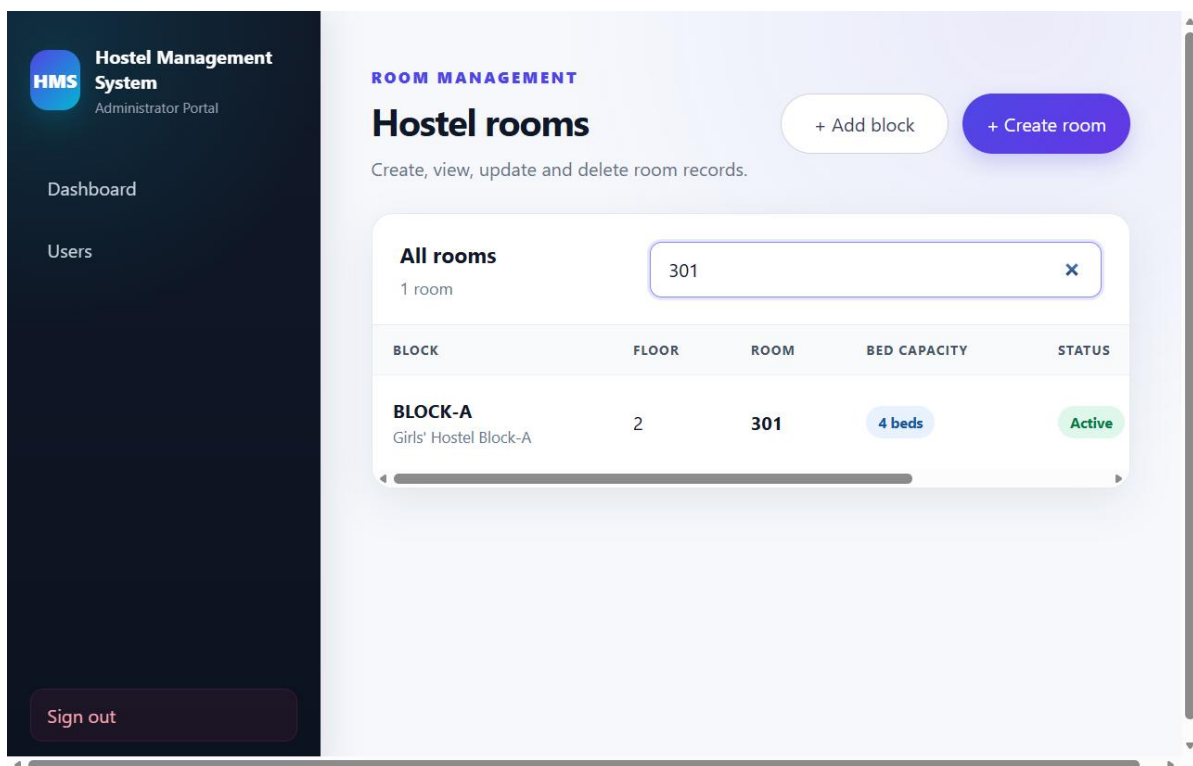
Block ID must belong to an active hostel block.

[Cancel](#) [Create room](#)

Evidence screenshot 41



Evidence screenshot 42



Evidence screenshot 43

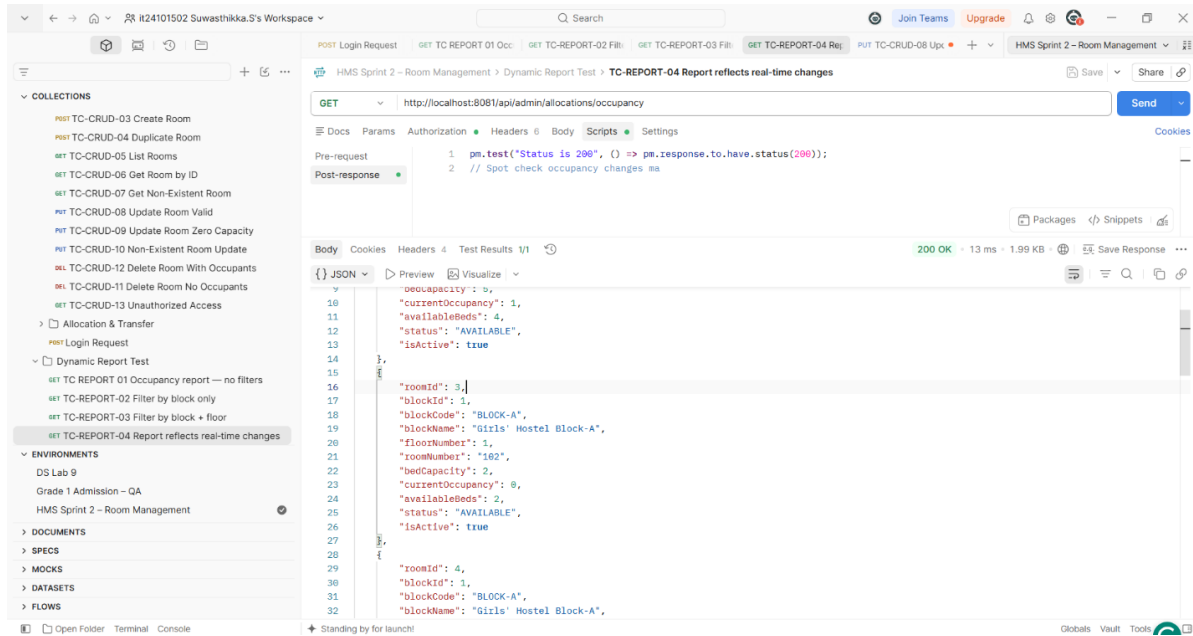
```
mysql> SELECT room_audit_log_id, administrator_user_id, room_id, action,
->         block_id, floor_number, room_number, bed_capacity, occurred_at
-> FROM room_audit_logs
-> WHERE room_id = 11
-> ORDER BY occurred_at;
```

room_audit_log_id	administrator_user_id	room_id	action	block_id	floor_number	room_number	bed_capacity	occurred_at
14	1	11	CREATE	2	4	405	3	2026-09-15 09:03:21.888790
15	1	11	UPDATE	2	4	405	3	2026-09-15 09:03:39.513085
16	1	11	DELETE	2	4	405	4	2026-09-15 09:04:09.789481

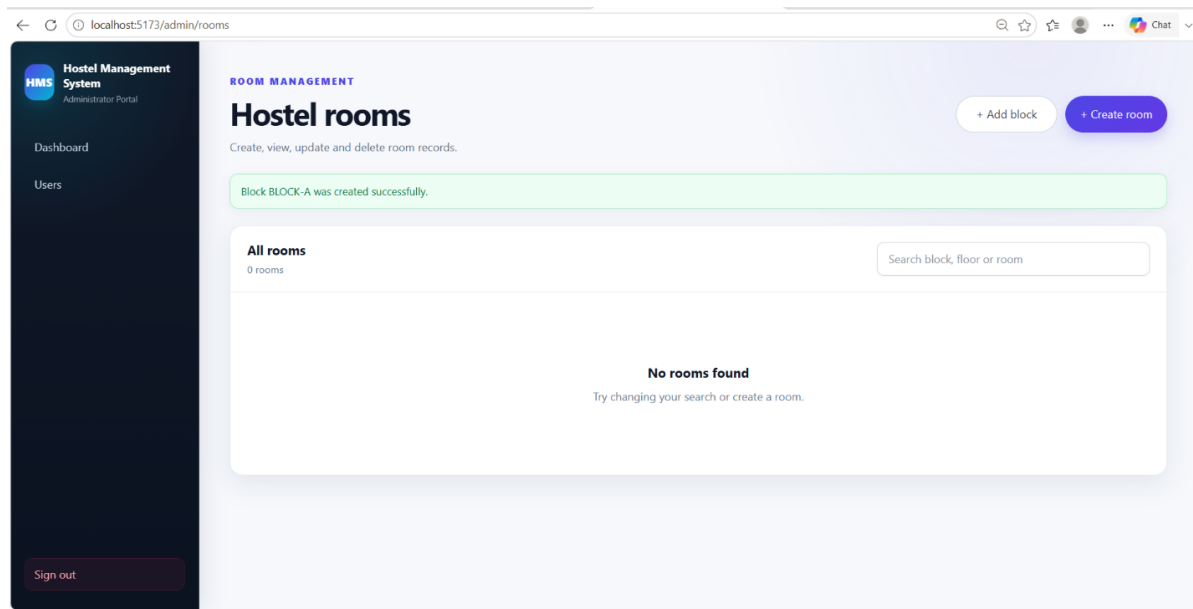
```
3 rows in set (0.00 sec)

mysql>
```

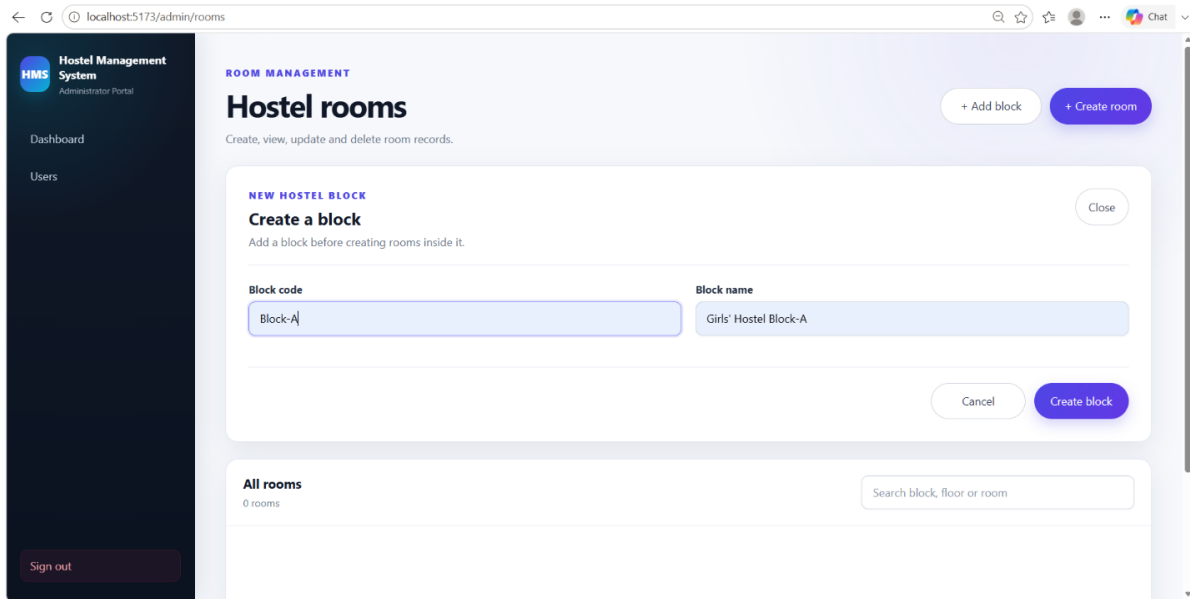
Evidence screenshot 44



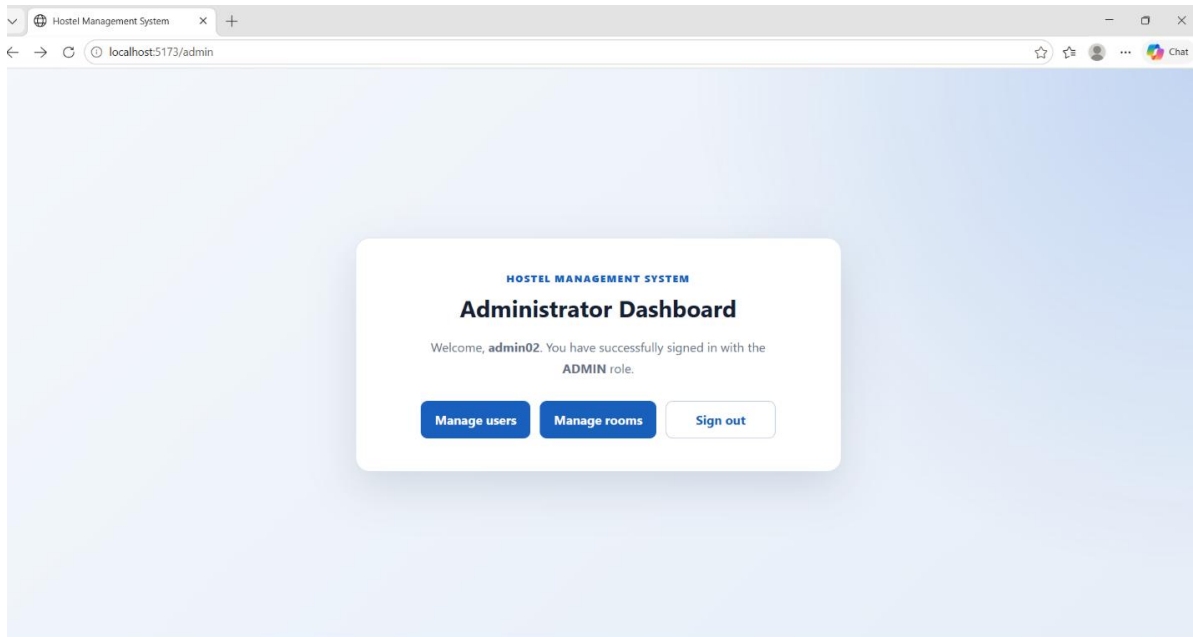
Evidence screenshot 45



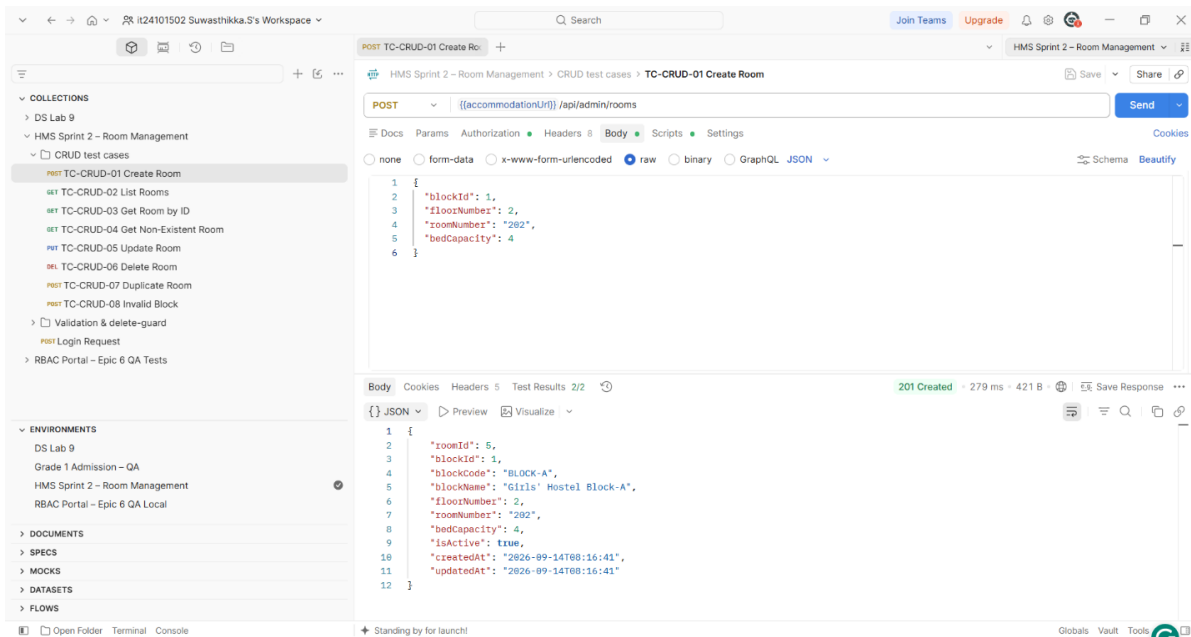
Evidence screenshot 46



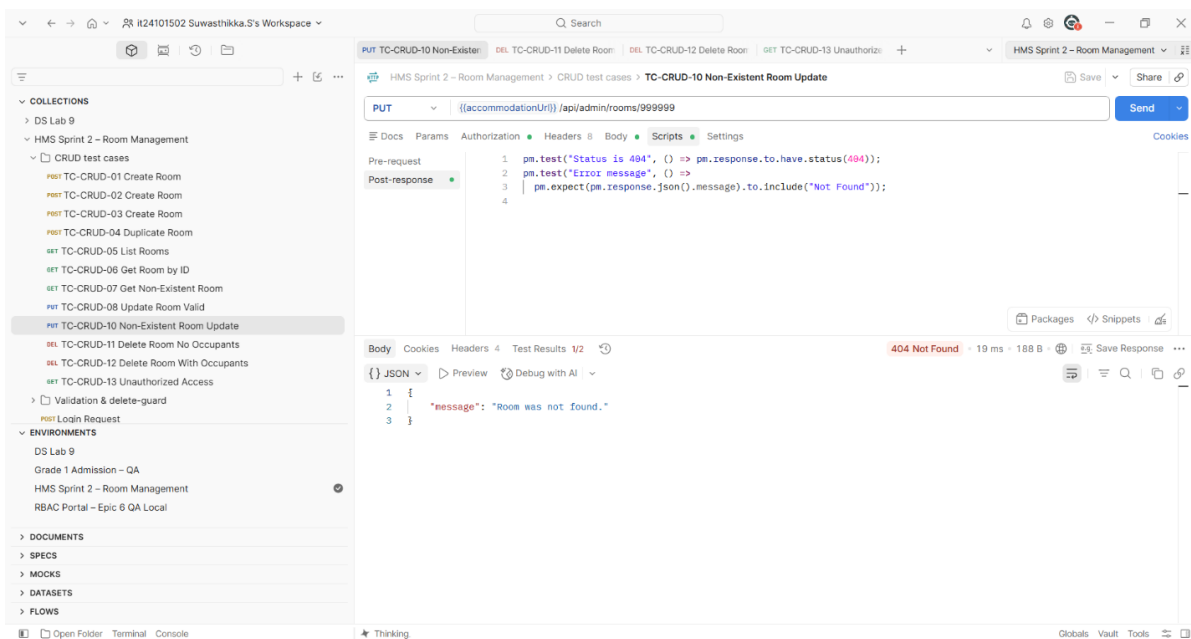
Evidence screenshot 47



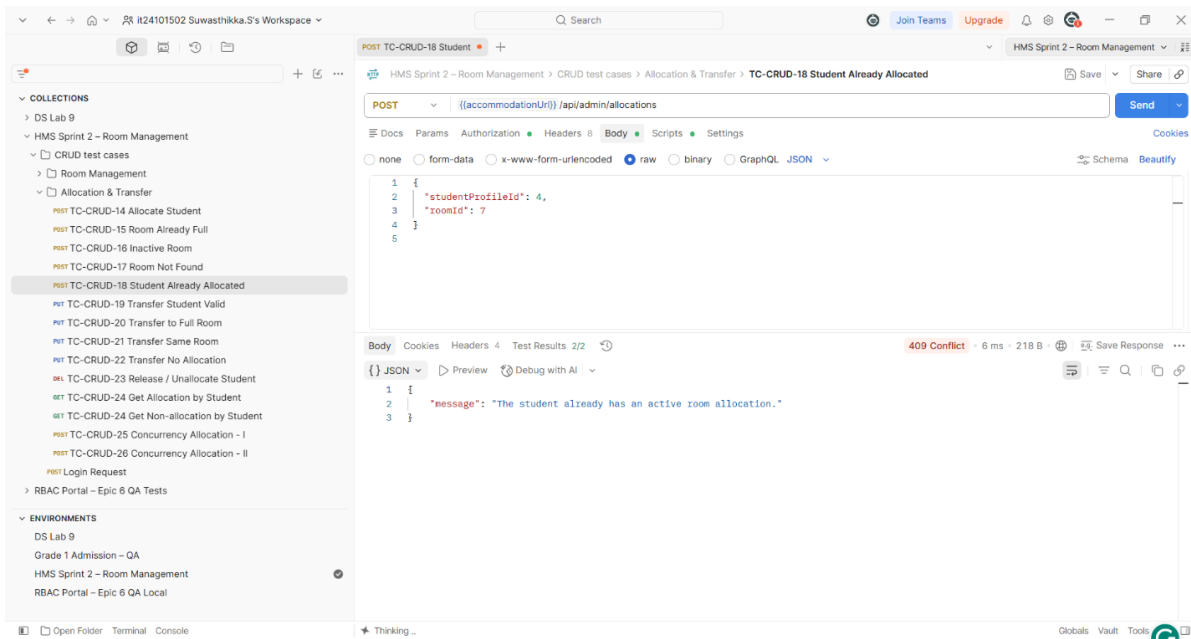
Evidence screenshot 48



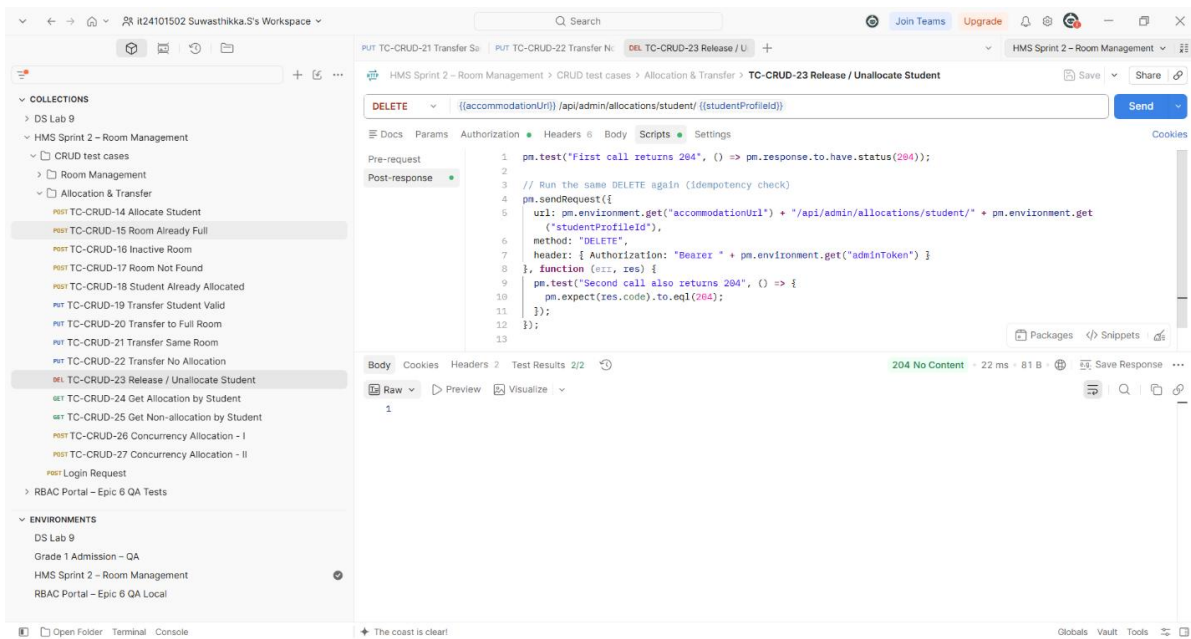
Evidence screenshot 49



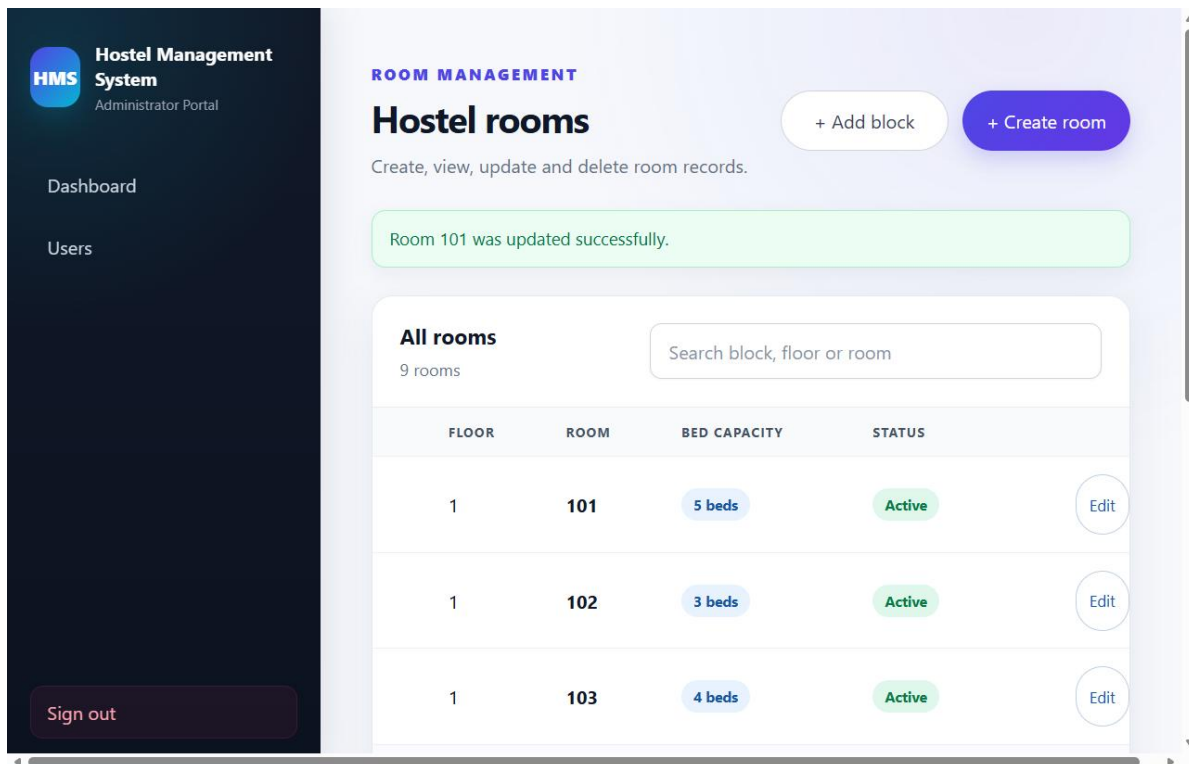
Evidence screenshot 50



Evidence screenshot 51



Evidence screenshot 52

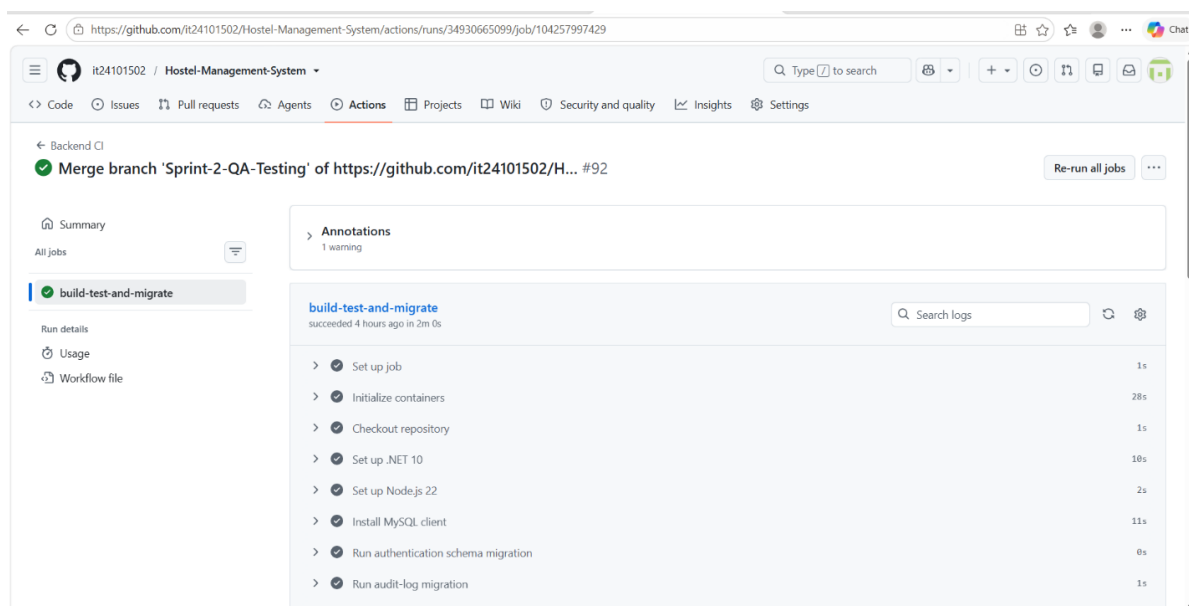


Evidence screenshot 53

```
PS C:\Users\Asus\Documents\Computer Science\Y3_S1\1. Case Study Project\Assignment\Sprint 2 QA\Hostel-Management-System> dotnet test tests\AccommodationService.Tests --filter RoomAllocationServiceTests
Restore complete (1.0s)
AccommodationService net10.0 succeeded (0.8s) -> backend\AccommodationService\bin\Debug\net10.0\AccommodationService.dll
AccommodationService.Tests net10.0 succeeded (0.3s) -> tests\AccommodationService.Tests\bin\Debug\net10.0\AccommodationService.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:00.19] Discovering: AccommodationService.Tests
[xUnit.net 00:00:00.27] Discovered: AccommodationService.Tests
[xUnit.net 00:00:00.33] Starting: AccommodationService.Tests
[xUnit.net 00:00:00.42] Finished: AccommodationService.Tests
AccommodationService.Tests test net10.0 succeeded (1.9s)

Test summary: total: 5, failed: 0, succeeded: 5, skipped: 0, duration: 1.9s
Build succeeded in 5.3s
PS C:\Users\Asus\Documents\Computer Science\Y3_S1\1. Case Study Project\Assignment\Sprint 2 QA\Hostel-Management-System> |
```

Evidence screenshot 54



Evidence screenshot 55

The screenshot shows the Postman interface with a workspace named "HMS Sprint 2 - Room Management". The left sidebar displays a collection of test cases under "CRUD test cases", with "TC-CRUD-05 List Rooms" selected. The main panel shows a GET request to the endpoint `[[accommodationUrl]] /api/admin/rooms`. The request has a "Pre-request" script and a "Post-response" script. The response is a 200 OK status with a JSON body containing an array of room details. The status bar at the bottom indicates "200 OK" and "25 ms" execution time.

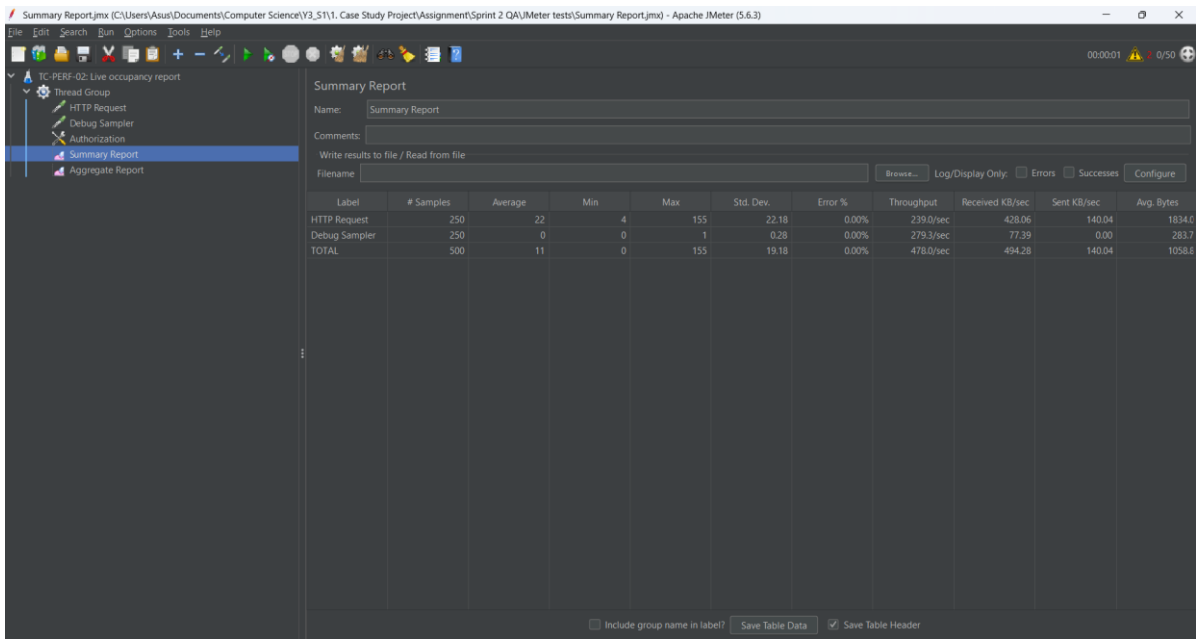
```
GET TC-CRUD-05 List Rooms GET TC-CRUD-06 Get Room by ID +
HMS Sprint 2 - Room Management > CRUD test cases > TC-CRUD-05 List Rooms
OET
Docs Params Authorization Headers 6 Body Scripts Settings
Pre-request
Post-response
1 pm.test("Status is 200", () => pm.response.to.have.status(200));
2 pm.test("Response is array", () => pm.expect(pm.response.json()).to.be.an("array"));
3 pm.expect(pm.response.json()).to.be.an("array");
4 pm.test("At least one room exists", () =>
5 pm.expect(pm.response.json().length).to.be.above(0));
6
Body Cookies Headers 4 Test Results 3/3
200 OK 25 ms 1.21 KB Save Response
1 {
2   {
3     "roomId": 1,
4     "blockId": 1,
5     "blockCode": "BLOCK-A",
6     "blockName": "Girls' Hostel Block-A",
7     "floorNumber": 1,
8     "roomNumber": "101",
9     "bedCapacity": 4,
10    "isActive": true,
11    "createdAt": "2026-09-13T07:23:07",
12    "updatedAt": "2026-09-13T07:23:07"
13  },
14  {
15    "roomId": 3,
16    "blockId": 1,
17    "blockCode": "BLOCK-A",
18    "blockName": "Girls' Hostel Block-A",
19    "floorNumber": 1,
20    "roomNumber": "102",
21    "bedCapacity": 3,
22    "isActive": true,
23    "createdAt": "2026-09-13T07:23:19",
24  }
25 }
```

Evidence screenshot 56

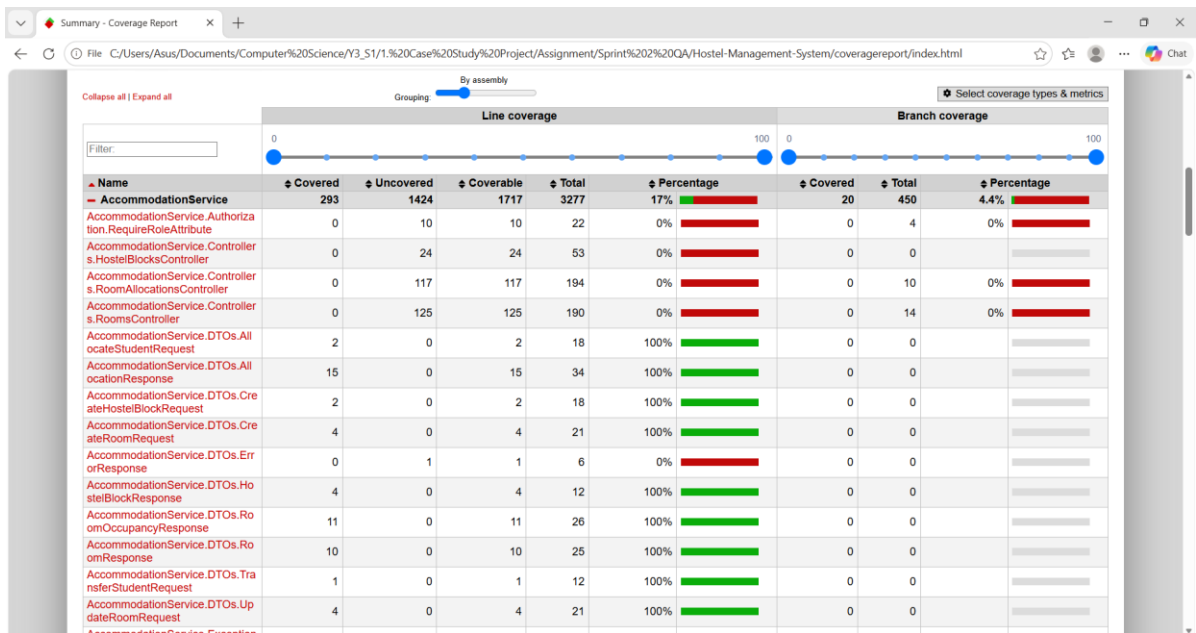
The screenshot shows the Postman interface with a workspace named "HMS Sprint 2 - Room Management". The left sidebar displays a collection of test cases under "Allocation & Transfer", with "TC-CRUD-26 Concurrency Allocation - I" selected. The main panel shows a POST request to the endpoint `[[accommodationUrl]] /api/admin/allocations`. The request has a "Body" tab selected, showing a JSON payload. The response is a 201 Created status with a JSON body containing allocation details. The status bar at the bottom indicates "201 Created" and "29 ms" execution time.

```
PUT TC-CRUD-21 Transfer TC-CRUD-22 Transfer TC-CRUD-23 Release TC-CRUD-24 Create TC-CRUD-25 Create POST TC-CRUD-26 Concurrency Allocation - I
HMS Sprint 2 - Room Management > Allocation & Transfer > TC-CRUD-26 Concurrency Allocation - I
POST
Docs Params Authorization Headers 8 Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL JSON
1 {
2   "studentProfileId": 5,
3   "roomId": 9
4 }
5
6
Body Cookies Headers 5 Test Results 1/2
201 Created 29 ms 530 B Save Response
1 {
2   "allocationId": 5,
3   "studentProfileId": 5,
4   "roomId": 9,
5   "blockId": 1,
6   "blockCode": "BLOCK-A",
7   "blockName": "Girls' Hostel Block-A",
8   "floorNumber": 2,
9   "roomNumber": "203",
10  "bedCapacity": 1,
11  "isActive": true,
12  "currentOccupancy": 1,
13  "availableBeds": 0,
14  "status": "FULL",
15  "allocatedAt": "2026-09-14T11:58:04",
16  "updatedAt": "2026-09-14T11:58:04"
17 }
```

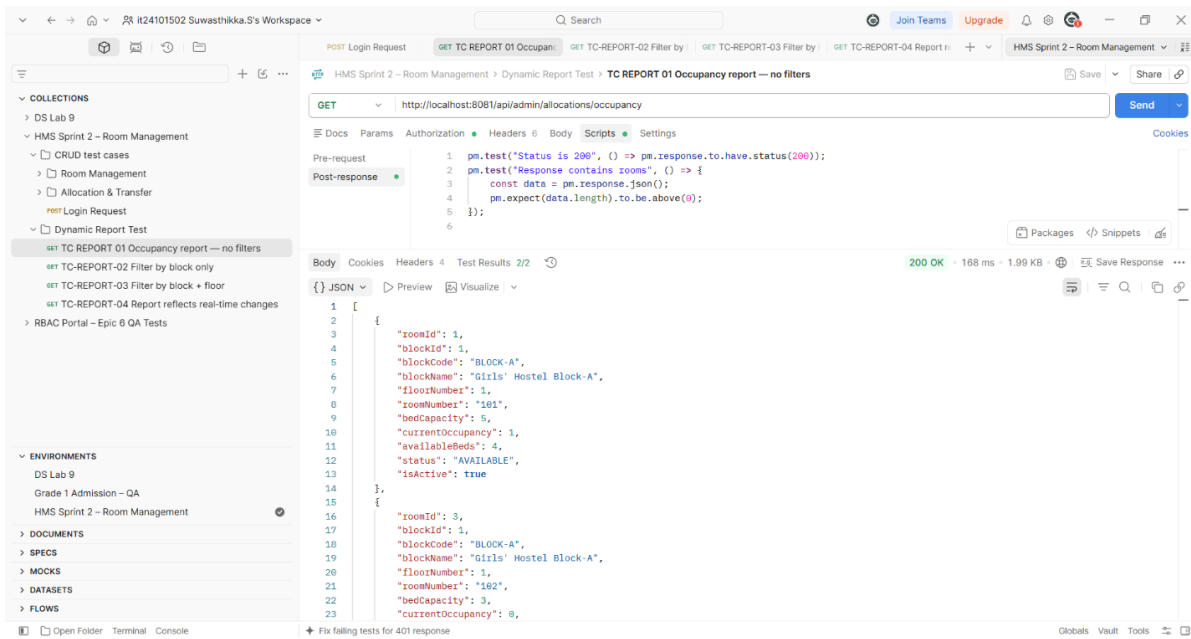
Evidence screenshot 57



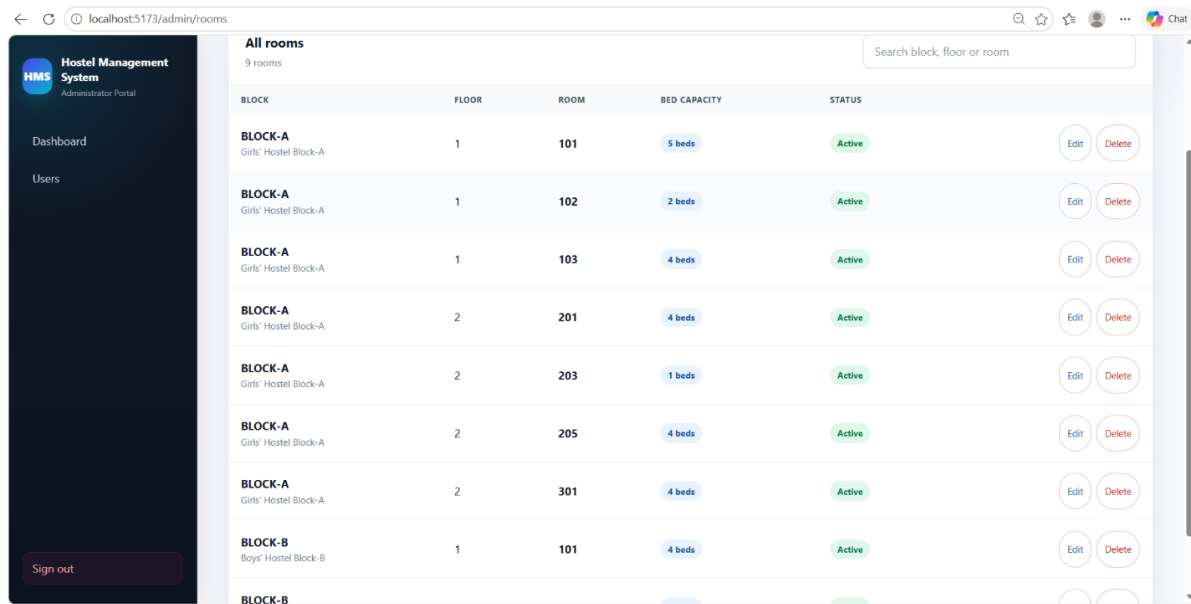
Evidence screenshot 58



Evidence screenshot 59



Evidence screenshot 60



Evidence screenshot 61

HMS

Hostel Management System

Administrator Portal

Dashboard

Users

Sign out

ROOM MANAGEMENT

Hostel rooms

Create, view, update and delete room records.

+ Add block

+ Create room

NEW ROOM

Close

Create a room

Enter the block, floor, room number and bed capacity.

Hostel block

BLOCK-A — Girls' Hostel Block-B

Floor number

1

Room number

101

Bed capacity

4

Block ID must belong to an active hostel block.

Cancel

Create room

All rooms

Search block, floor or room

Evidence screenshot 62

HMS

Hostel Management System

Accommodation Operations

Dashboard

Rooms

Users

Allocations

Sign out

ROOMS

4

Matching filters

TOTAL BEDS

15

Maximum capacity

OCCUPIED BEDS

0

Current allocations

AVAILABLE BEDS

15

Ready for allocation

NEW ALLOCATION

Close

Allocate a student

Only active rooms with available beds are shown.

Student profile ID

2

Destination room

BLOCK-A · Floor 1 · Room 101 · 4 free

Cancel

Allocate student

Live occupancy

Real-time room availability by block and floor.

All blocks

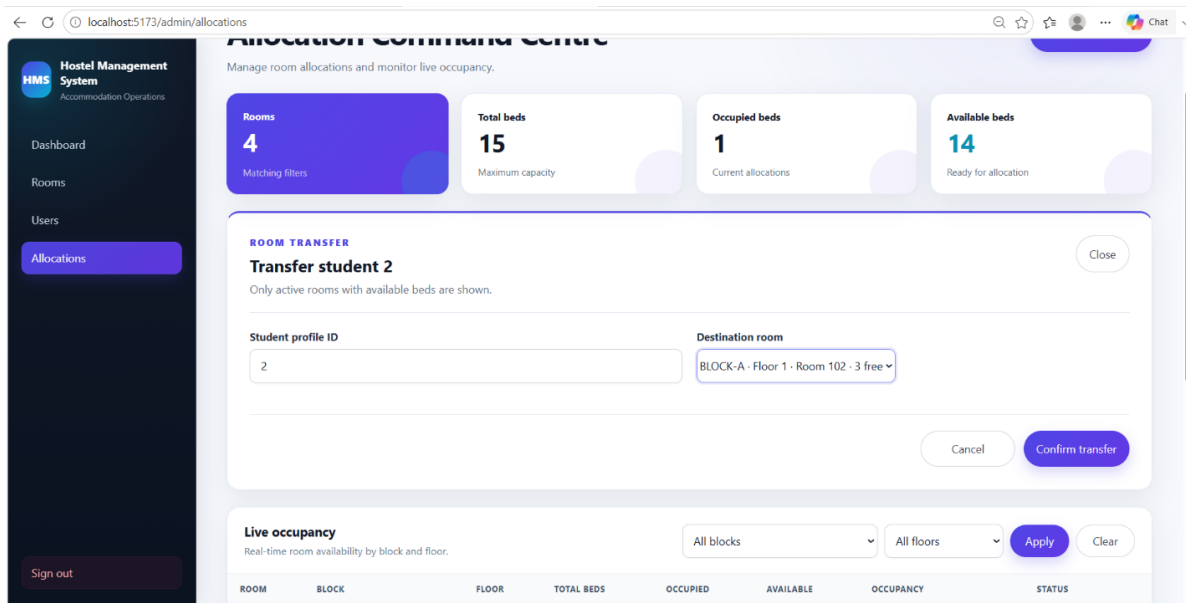
All floors

Apply

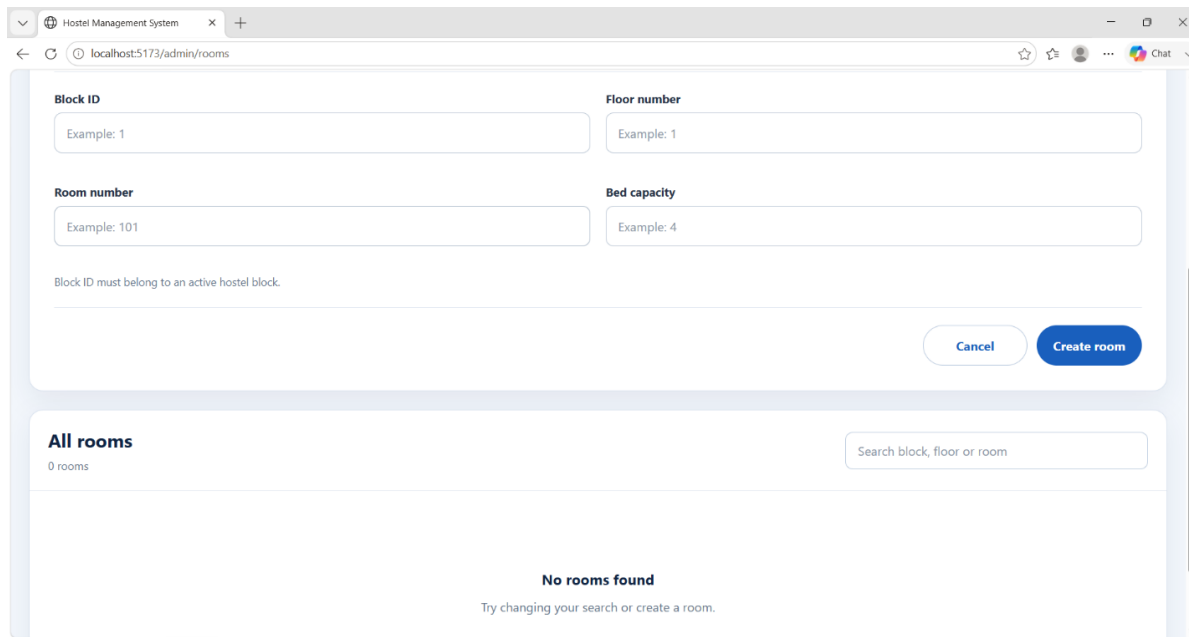
Clear

ROOM	BLOCK	FLOOR	TOTAL BEDS	OCCUPIED	AVAILABLE	OCCUPANCY	STATUS
101	BLOCK-A Girls' Hostel Block-A	1	4	0	4	0%	AVAILABLE
102	BLOCK-A	1	3	0	3	0%	AVAILABLE

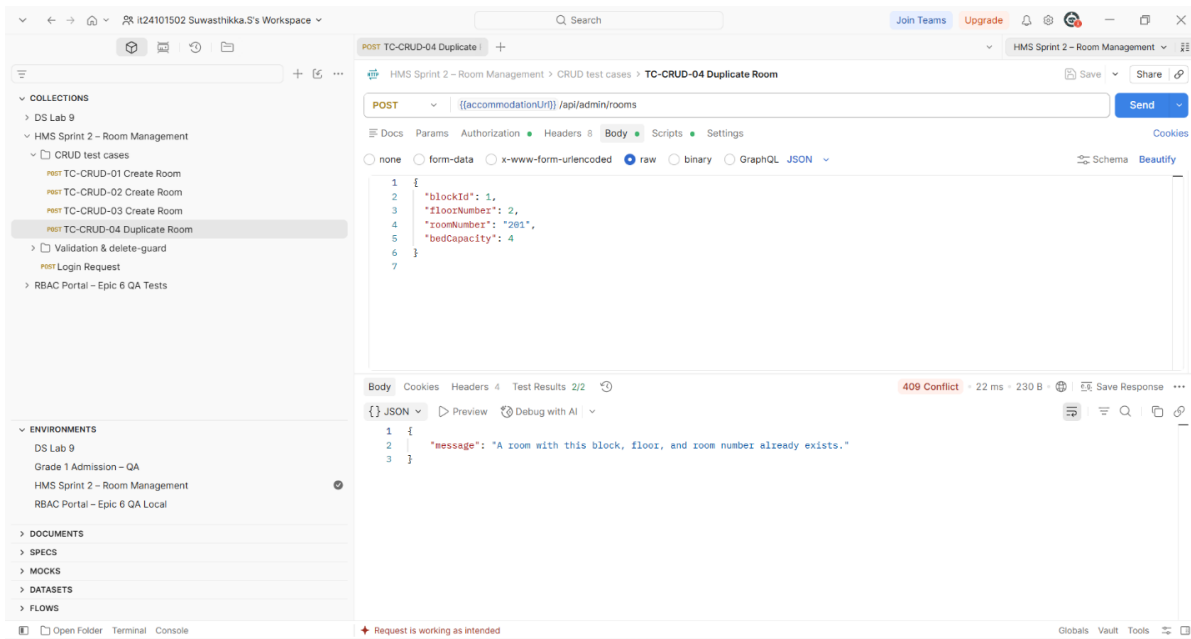
Evidence screenshot 63



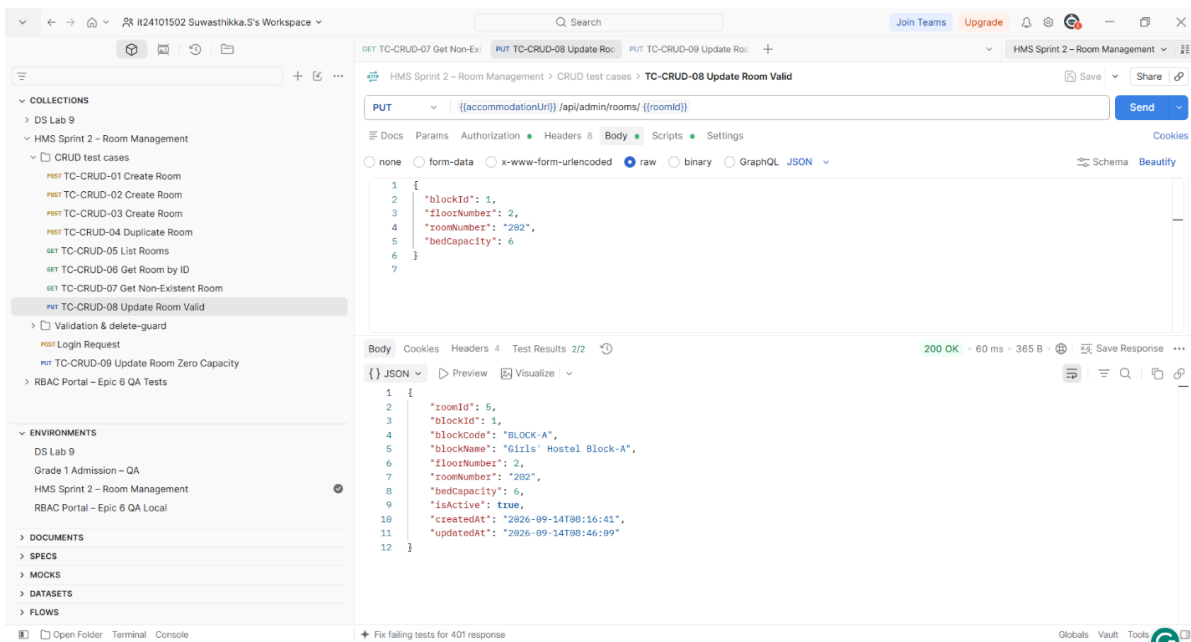
Evidence screenshot 64



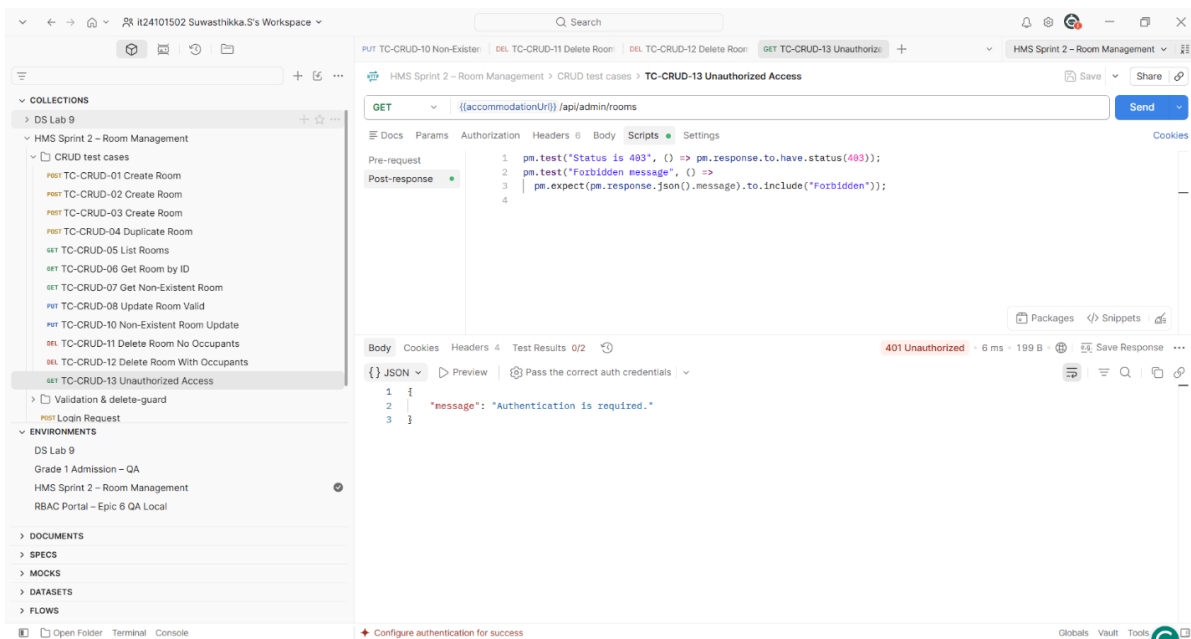
Evidence screenshot 65



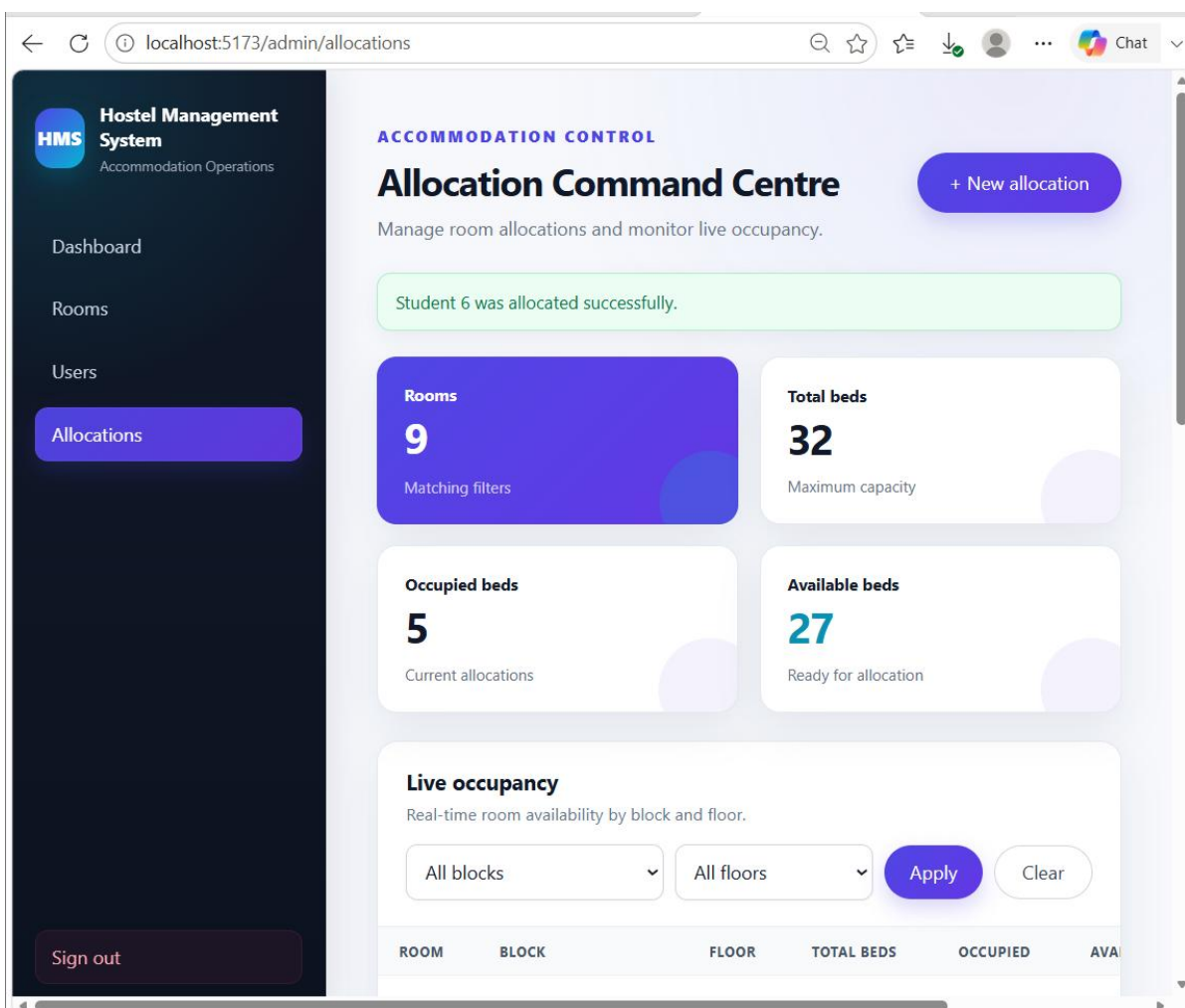
Evidence screenshot 66



Evidence screenshot 67



Evidence screenshot 68



Evidence screenshot 69

**Hostel Management System**
Administrator Portal

Dashboard

Users

Sign out

ROOM MANAGEMENT

Hostel rooms

Create, view, update and delete room records.

+ Add block

Create room

NEW ROOM

Close

Create a room

Enter the block, floor, room number and bed capacity.

Hostel block

Floor number

Room number

Bed capacity

BLOCK-A — Girls' Hostel Block-A

Example: 1

Example: 101

0

Room number is required.

Bed capacity must be a positive whole number.

Block ID must belong to an active hostel block.

Evidence screenshot 70

```
PS C:\Users\Asus\Documents\Computer Science\Y3_S1\1. Case Study Project\Assignment\Sprint 2 QA\Hostel-Management-System> dotnet test tests\AccommodationService.Tests --filter HostelBlockServiceTests
Restore complete (0.8s)
AccommodationService net10.0 succeeded (0.6s) → backend\AccommodationService\bin\Debug\net10.0\AccommodationService.dll
AccommodationService.Tests net10.0 succeeded (0.3s) → tests\AccommodationService.Tests\bin\Debug\net10.0\AccommodationService.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:00.14] Discovering: AccommodationService.Tests
[xUnit.net 00:00:00.21] Discovered: AccommodationService.Tests
[xUnit.net 00:00:00.25] Starting: AccommodationService.Tests
[xUnit.net 00:00:00.34] Finished: AccommodationService.Tests
AccommodationService.Tests test net10.0 succeeded (1.7s)

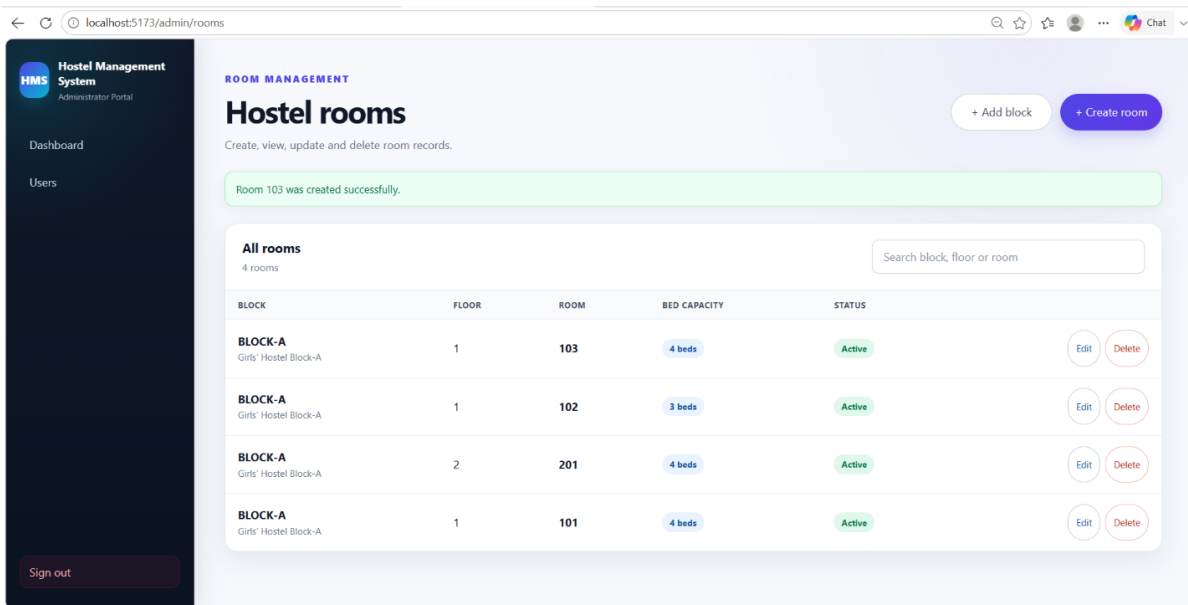
Test summary: total: 3, failed: 0, succeeded: 3, skipped: 0, duration: 1.7s
Build succeeded in 4.4s
PS C:\Users\Asus\Documents\Computer Science\Y3_S1\1. Case Study Project\Assignment\Sprint 2 QA\Hostel-Management-System> |
```

Evidence screenshot 71

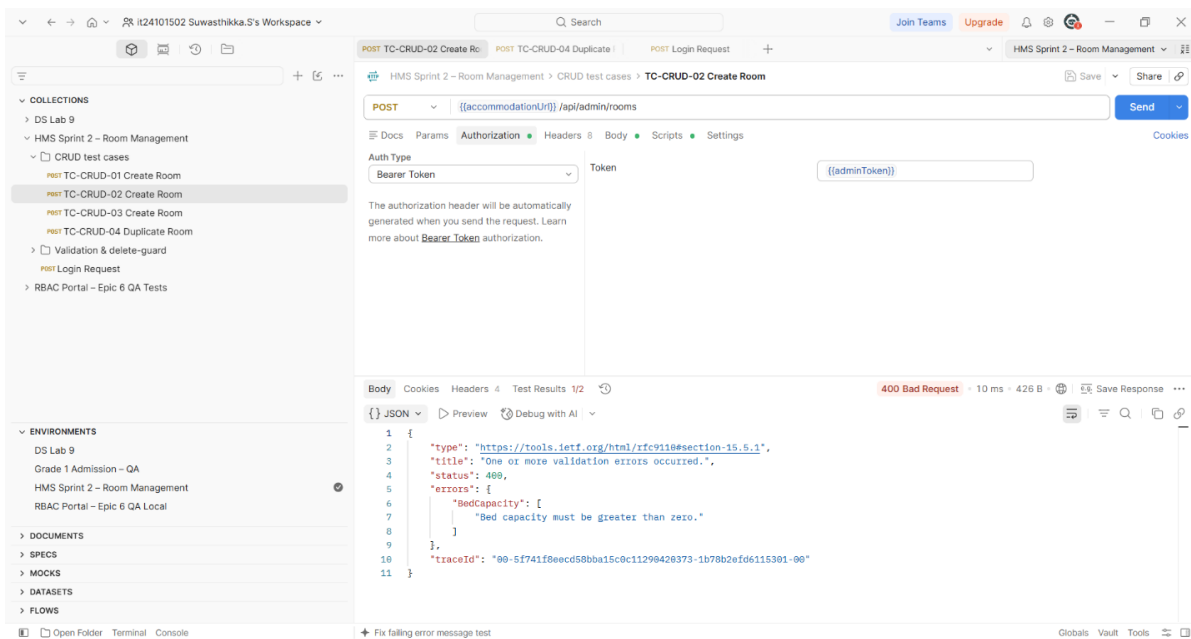
```
mysql> SELECT allocation_audit_log_id, allocation_id, administrator_user_id, student_profile_id, action, from_room_id, to_room_id, occurred_at FROM student_room_allocation_audit_logs WHERE student_profile_id = 4 ORDER BY occurred_at;
+-----+-----+-----+-----+-----+-----+-----+-----+
| allocation_audit_log_id | allocation_id | administrator_user_id | student_profile_id | action | from_room_id | to_room_id | occurred_at |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 6 | 4 | 1 | 4 | ALLOCATE | NULL | 8 | 2026-09-14 11:42:55.417077 |
| 13 | 4 | 1 | 4 | TRANSFER | 8 | 6 | 2026-09-15 09:07:29.886793 |
| 14 | 4 | 1 | 4 | RELEASE | 6 | NULL | 2026-09-15 09:08:00.200237 |
+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```

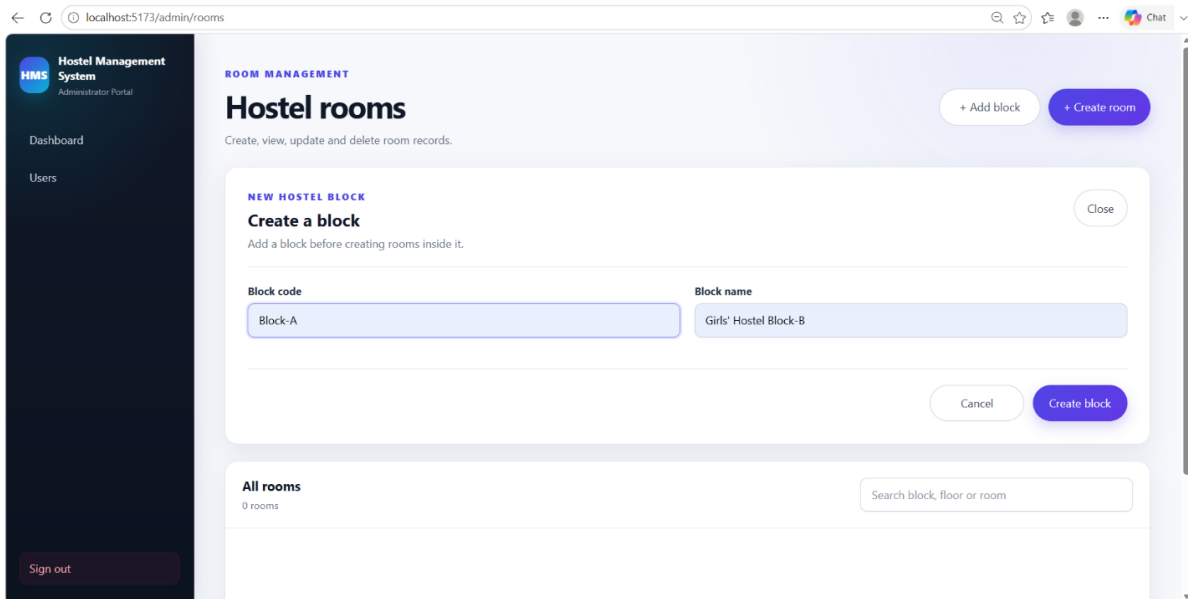
Evidence screenshot 72



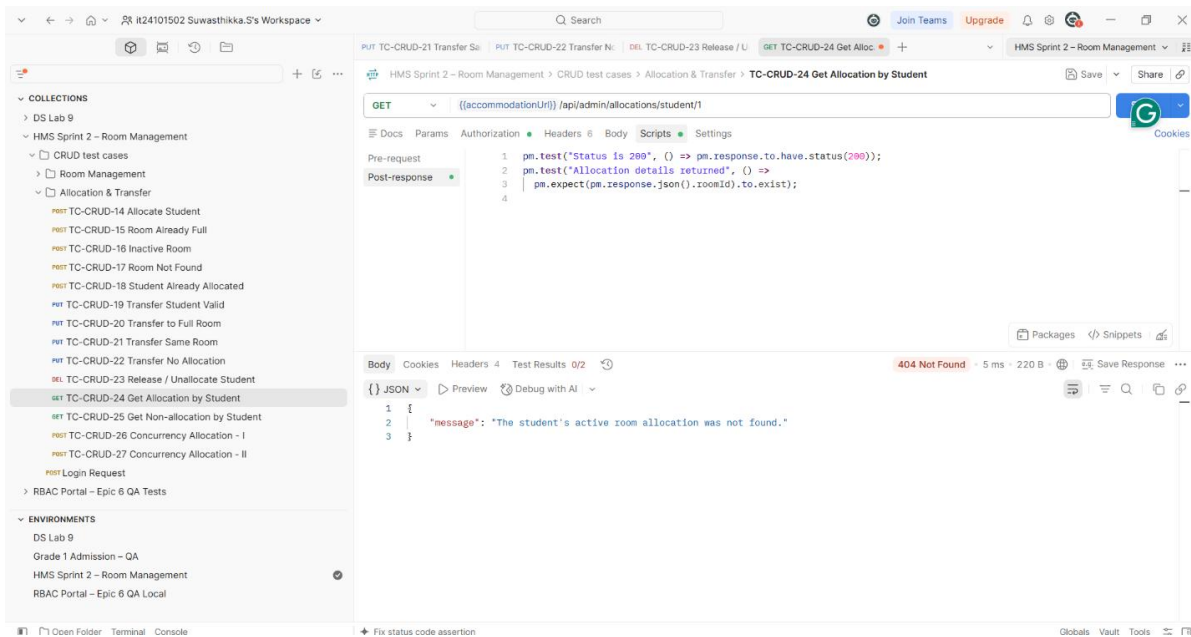
Evidence screenshot 73



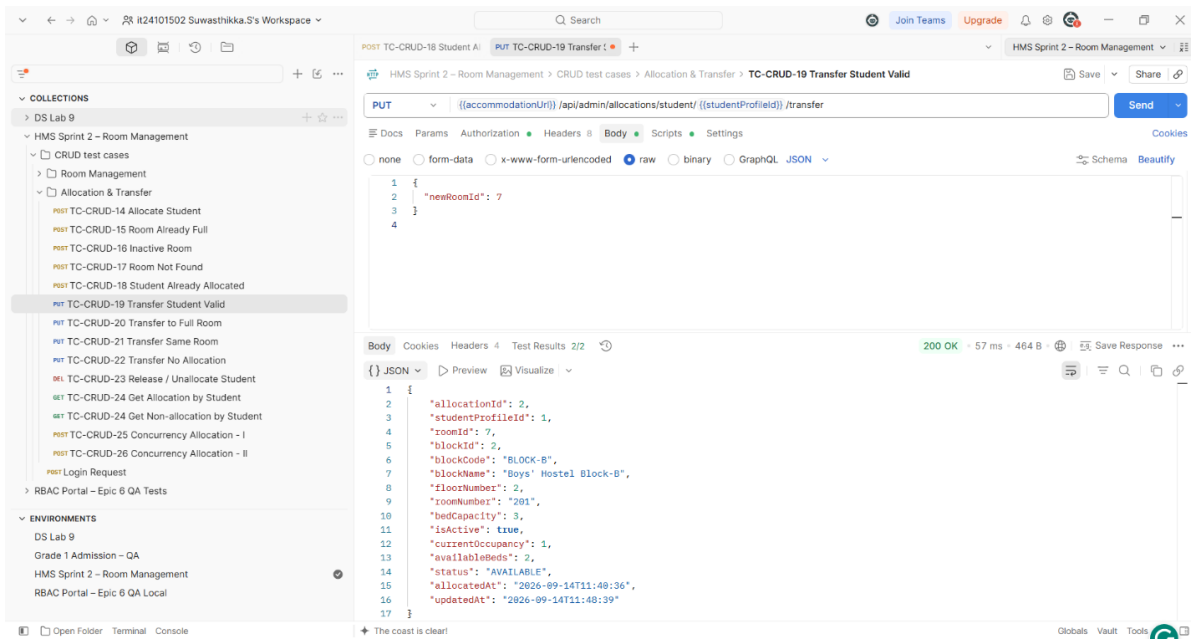
Evidence screenshot 74



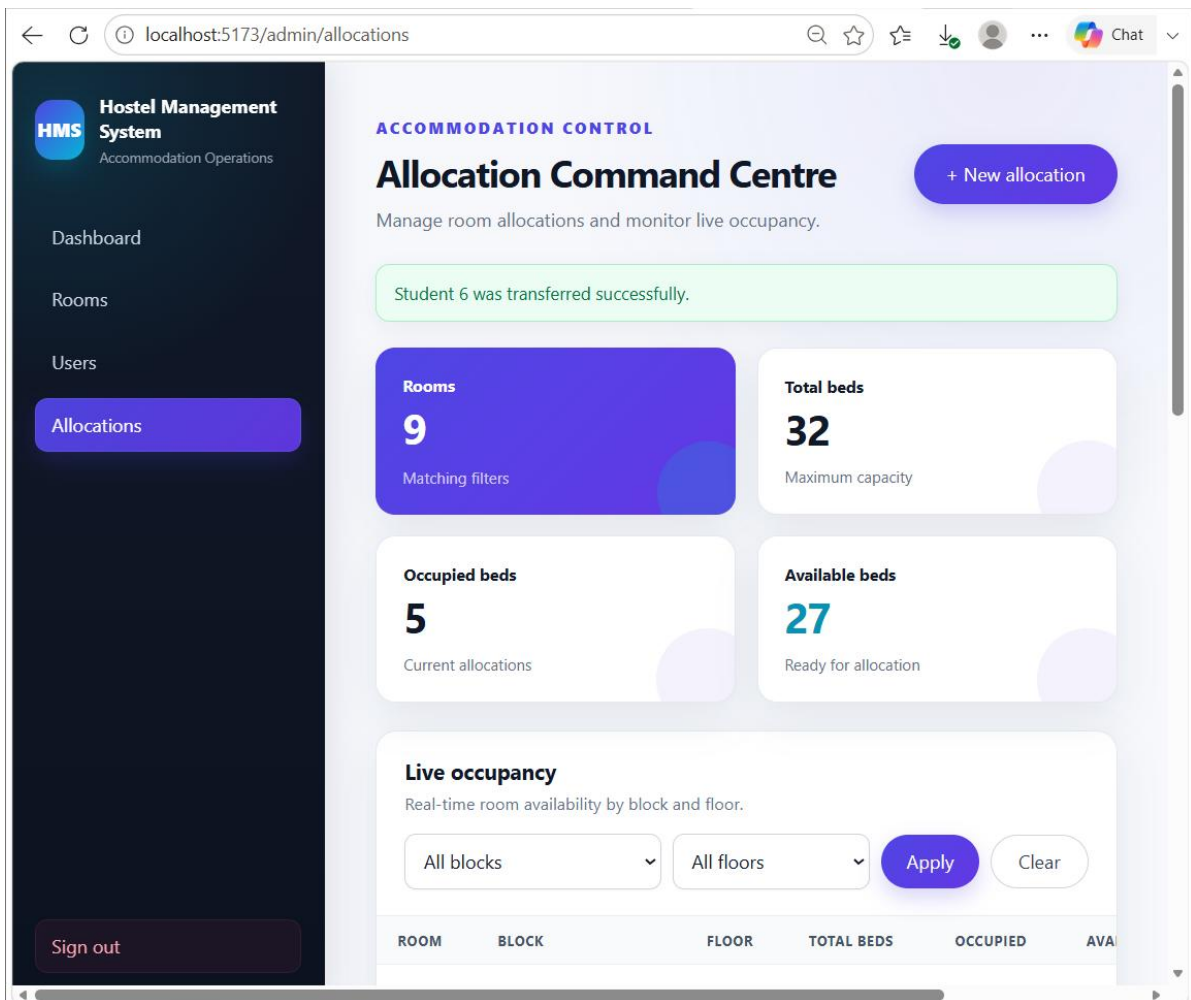
Evidence screenshot 75



Evidence screenshot 76



Evidence screenshot 77



Evidence screenshot 78

Summary Report.jmx (C:\Users\Asus\Documents\Computer Science\Y2_S1\1. Case Study Project\Assignment\Sprint 2 QA\Meter tests\Summary Report.jmx) - Apache JMeter (5.6.3)

TC-PERF-02: Live occupancy report

- Thread Group
 - HTTP Request
 - Debug Sampler
 - Authorization
 - Summary Report
 - Aggregate Report

Aggregate Report

Name: /Aggregate Report

Comments:

Write results to file / Read from file

Filename: ☐ Log/Display Only ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
HTTP Request	250	22	15	44	59	134	4	155	0.00%	239.0/sec	428.06	140.04
Debug Sampl...	250	0	0	0	1	1	0	1	0.00%	279.3/sec	77.39	0.00
TOTAL	500	11	1	31	44	86	0	155	0.00%	478.0/sec	494.28	140.04

☐ Include group name in label? ☒ Save Table Header

Evidence screenshot 79

Hostel Management System

localhost:5173/admin/rooms

ROOM MANAGEMENT

Hostel rooms

Create, view, update and delete room records.

[+ Create room](#)

NEW ROOM

Create a room [Close](#)

Enter the block, floor, room number and bed capacity.

Block ID

Floor number

Room number

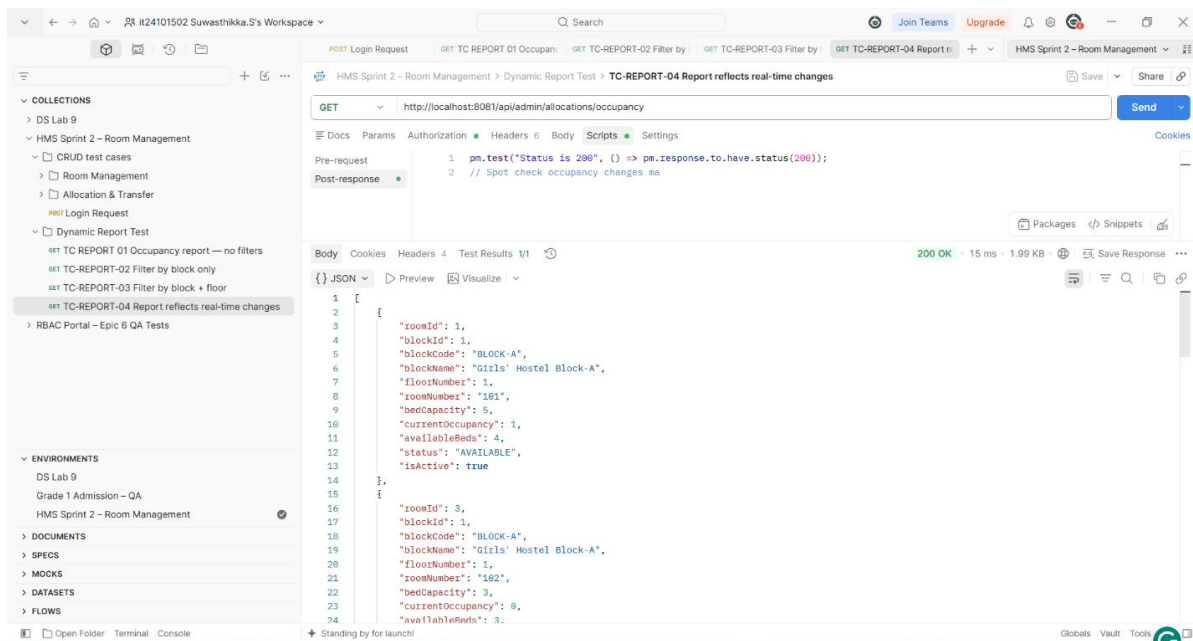
Bed capacity

Block ID must belong to an active hostel block.

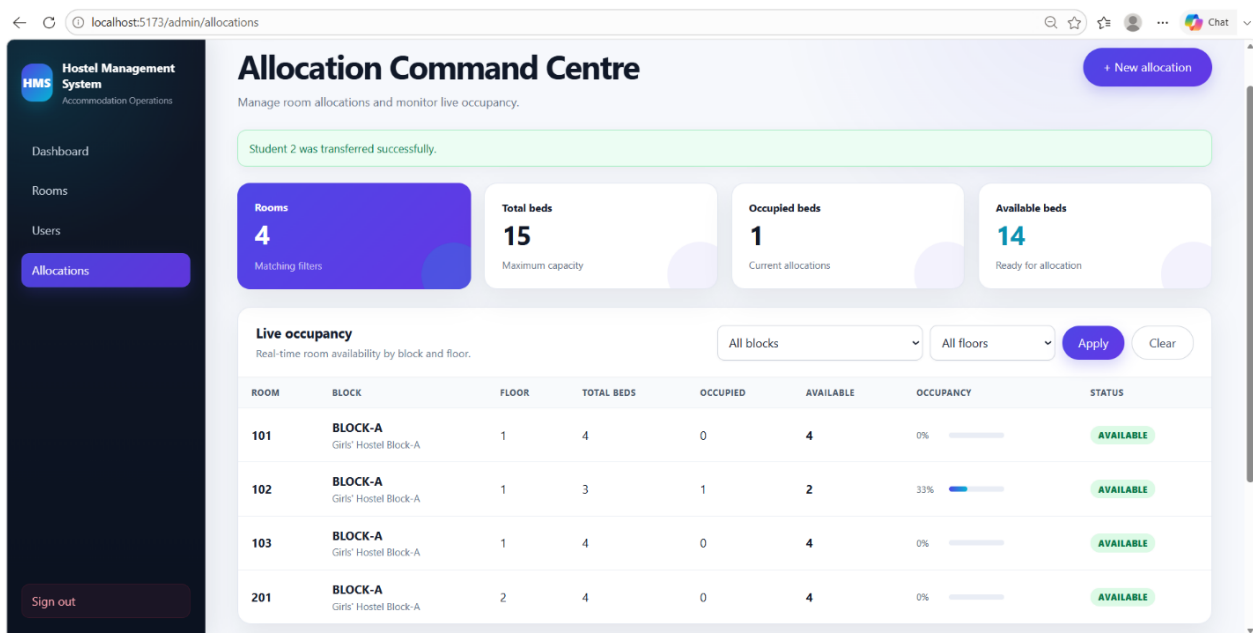
[Cancel](#)

[Create room](#)

Evidence screenshot 80



Evidence screenshot 81



Evidence screenshot 82